UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
CENTRO DE TECNOLOGIA
GRADUAÇÃO EM ENGENHARIA DE COMPUTAÇÃO

Ciência de Dados Aplicada à Saúde Ocupacional: Monitoramento e Análise de Afastamentos de Servidores Públicos Atendidos pela Unidade SIASS da UFRN

Alisson Sousa Moreira

Orientador: Prof. Dr. Luiz Affonso Henderson Guedes De Oliveira

Natal, RN, 20 de janeiro de 2025

UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
CENTRO DE TECNOLOGIA
GRADUAÇÃO EM ENGENHARIA DE COMPUTAÇÃO

Ciência de Dados Aplicada à Saúde Ocupacional: Monitoramento e Análise de Afastamentos de Servidores Públicos Atendidos pela Unidade SIASS da UFRN

Alisson Sousa Moreira

Orientador: Prof. Dr. Luiz Affonso Henderson Guedes De Oliveira

Co-orientador:

Trabalho de Conclusão de Curso de Graduação na modalidade Monografia, submetido como parte dos requisitos necessários para conclusão do curso de Engenharia de Computação pela Universidade Federal do Rio Grande do Norte (UFRN/CT).

Natal, RN, 20 de janeiro de 2025

Universidade Federal do Rio Grande do Norte - UFRN
Sistema de Bibliotecas - SISBI
Catalogação de Publicação na Fonte. UFRN - Biblioteca Central Zila Mamede

Moreira, Alisson Sousa.
Ciência de dados aplicada à saúde ocupacional: monitoramento e análise de afastamentos de servidores Públicos atendidos pela Unidade SIASS da UFRN / Alisson Sousa Moreira. – 2025.
44f.: il.

Monografia (Graduação) – Universidade Federal do Rio Grande do Norte, Departamento de Engenharia de Computação e Automação, Engenharia de Computação, Natal, 2025.
Orientação: Dr. Luiz Affonso Henderson Guedes De Oliveira.

1. Streamlit – Monografia. 2. SIASS – Monografia. 3. Atendimentos – Monografia. 4. Sistema – Monografia. 5. Algoritmo – Monografia. I. Oliveira, Luiz Affonso Henderson Guedes De. II. Título.

RN/UF/BCZM CDU 004

Elaborado por Raimundo Muniz de Oliveira – CRB-15/429

Ciência de Dados Aplicada à Saúde Ocupacional: Monitoramento e Análise de Afastamentos de Servidores Públicos Atendidos pela Unidade SIASS da UFRN

Alisson Sousa Moreira

Monografia aprovada em 20 de janeiro de 2025, pela banca examinadora composta pelos seguintes membros:

Prof. Dr. Luiz Affonso Henderson Guedes De Oliveira (orientador) . . . . . .
DCA/UFRN

Eng. Felipe Lima . . . . . DCA/UFFN

Eng. Lemysom Lemos . . . . . DCA/UFRN

Agradecimentos

Ao meu orientador, professor Luiz Affonso Henderson Guedes De Oliveira, sou grato pela orientação e pela compreenção.

Aos meus amigos Leonado, Érika, Guilherme, Rafael e em especial a Vanuza, por terem acreditado em mim, pelo apoio e conversas nos momentos que precisei. Aos meus colegas de graduação Wellington, Jadson, Gustavo, Ricardo e João Lucas, que apesar de não termos continuado juntos no segundo ciclo foram de grande ajuda no inicio da gradução.

À minha mãe, Antônia, pelo apoio, paciência e por sempre ter acreditado em mim.

À minha mulher e grande amor da minha vida, Rafaela Campos, que me incentivou a continua apesar de todos os problema que passei. Sem ela eu não teria conseguido.

Resumo

Este trabalho apresenta o desenvolvimento de um sistema interativo utilizando Python e Streamlit, destinado ao uso interno do SIASS na Universidade Federal do Rio Grande do Norte (UFRN). O sistema gera gráficos em tempo real baseados nos atendimentos realizados pela Divisão de Atenção à Saúde do Servidor (DAS), possibilitando análises quantitativas de tipos de perícia realizadas, comparações entre períodos e proporções entre os atendimentos. A solução busca aprimorar a gestão de dados relacionados à homologação de atestados médicos, oferecendo suporte à tomada de decisões administrativas. Ao integrar tecnologia e gestão, este projeto contribui para a eficiência e transparência dos processos internos.

Palavras-chave: Streamlit, SIASS, atendimentos, sistema, algoritmo.

Abstract

This study presents the development of an interactive system using Python and Streamlit, designed for internal use by the SIASS at the Federal University of Rio Grande do Norte (UFRN). The system generates real-time charts based on the services provided by the Division of Health Care for Servers (DAS), enabling quantitative analyses of the types of evaluations conducted, comparisons between periods, and proportions among service types. The solution aims to improve the management of data related to the homologation of medical certificates, providing support for administrative decision-making. By integrating technology and data management, this project contributes to the efficiency and transparency of internal processes.

Keywords: Streamlit, SIASS, services, system, algorithm.

Sumário

Sumário **i**

Lista de Figuras **iii**

Lista de Tabelas **v**

1 Introdução **1**

2 Fundamentação Teórica **5**

2.1 Pandas 5
2.2 Pygsheets 5
2.3 Datetime 5
2.4 Plotly 6
2.5 Matplotlib 6
2.6 Streamlit 6
2.7 st_pages 6
2.8 Classes em python 6
2.9 Apps Script 7
2.10 Google Cloud 7
2.11 Integração das Tecnologias 7

3 Metodologia/Implementação **9**

3.1 Interface 10
3.2 Aquisição dos dados 11
3.3 Funções 14
3.4 Classes 18
3.5 Tratamento de dados 28
3.6 Mapeamento de combinações 29

4 Experimentos e Resultados **33**

5 Conclusão **41**

Lista de Figuras

3.1 Trecho do algoritmo para a selectbox do Streamlit. 10
3.2 Trecho do algoritmo para a criação da navegação entre as páginas. 11
3.3 Algoritmo do Apps Script responsável pela coleta de dados da agenda e armazenamento em uma planilha do Google. 12
3.4 Exemplo de algoritmo para receber dados de planilhas em tempo real. . . 13
3.5 Tratamento de dados inicial do DataFrame com as planilhas. 14
3.6 Função de filtro de datas. 15
3.7 Primeira parte da função para receber e preservar as entradas de datas. . . 16
3.8 Segunda parte da função para receber e preservar as entradas de datas. . . 16
3.9 Função para preencher as datas que não houveram atendimento. 17
3.10 Função para subtrair um mês da data recebida. 18
3.11 Construtor da classe *Gráfico*. 20
3.12 Método da classe *Gráfico* para o gráfico de linha. 20
3.13 Método da classe *Gráfico* para o gráfico de coluna. 21
3.14 Método da classe *Gráfico* para o gráfico de barra. 21
3.15 Método da classe *Gráfico* para tabelas. 22
3.16 Método da classe *Gráfico* para quando não existe visualização para determinada combinação. 22
3.17 Método da classe *Gráfico* para o gráfico de pizza de proporção geral. . . . 23
3.18 Método da classe *Gráfico* para o gráfico de coluna para porporção geral. . 23
3.19 Método da classe *Gráfico* para o gráfico de barra para porporção geral. . . 24
3.20 Método da classe *Gráfico* para o gráfico de pizza comparando períodos de um mesmo tipo de atendimento. 25
3.21 Construtor da classe *MultGráfico*. 26
3.22 Método da classe *MultGráfico* para o gráfico de linha de proporção geral. 26
3.23 Método da classe *MultGráfico* para o gráfico de linha. 27
3.24 Método da classe *MultGráfico* para o gráfico de coluna. 27
3.25 Método da classe *MultGráfico* para o gráfico de barra. 28
3.26 Exemplo de tratamento de dados. 28
3.27 Mapeamento da combinações para os gráficos de junta médica. 30
3.28 Mapeamento da combinações para os gráficos de perícia singular. 30
3.29 Mapeamento da combinações para os gráficos de porporção geral. 30
3.30 Mapeamento da combinações para os gráficos de perícia documental. . . 31
3.31 Mapeamento da combinações para os gráficos de perícia odontológica. . . 31
3.32 Trecho do algoritmo para determinar qual gráfico será exibido de acordo com a combinação. 32

4.1 Tela inicial do sistema. 34
4.2 Barra lateral com sitema de navegação para multiplas páginas e um input de datas. 34
4.3 Gráfico de linha para a proporção geral dos atendimentos sem comparação com o periodo anterior. 35
4.4 Gráfico de linha para atendimento de perícia singular sem comparação com o periodo anterior. 35
4.5 Gráfico de coluna para atendimento de perícia documental comparando com "x"dias anteriores. 36
4.6 Gráfico de coluna para atendimento de junta médica comparando com o mês anterior. 37
4.7 Gráfico de barra para atendimento de perícia odontológica sem comparação com periodo anterior. 37
4.8 Gráfico de pizza para atendimento de perícia singular comparando com "x"dias anteriores. 38
4.9 Gráfico de linha para atendimento de perícia documental comparando com o mês anterior. 38
4.10 Tabela para atendimentos de perícia singular. 39
4.11 Mensagem exibida para combinações que não geram gráficos. 39

Lista de Tabelas

Capítulo 1

Introdução

A gestão eficiente de processos administrativos relacionados à saúde dos servidores públicos é essencial para garantir o bem-estar dos trabalhadores e a continuidade dos serviços prestados à sociedade. Nesse contexto, o Subsistema Integrado de Atenção à Saúde do Servidor (SIASS) desempenha um papel fundamental, ao gerenciar as demandas de homologação de atestados médicos e regulamentação de afastamentos no âmbito da Administração Pública Federal. O Manual de Perícia Oficial em Saúde do Servidor Público Federal destaca a importância das avaliações periciais para assegurar a conformidade dos afastamentos com as normativas legais e o correto cômputo das licenças, considerando critérios técnicos e legais [1].

Na Universidade Federal do Rio Grande do Norte (UFRN), a Divisão de Atenção à Saúde do Servidor (DAS), onde está integrado o SIASS, centraliza as atividades relacionadas à homologação de atestados. Essa unidade oferece suporte aos servidores que necessitam de afastamentos por motivos de saúde, desempenhando um papel crucial no funcionamento administrativo [2][3]. Contudo, o volume de atendimentos, a diversidade dos casos e a necessidade de análises técnicas detalhadas tornam esse processo desafiador. Além disso, a falta de ferramentas específicas para análise e visualização de dados em tempo real pode dificultar a identificação de padrões, a comparação de períodos e a geração de informações estratégicas para a gestão.

Os atendimentos realizados pelo SIASS seguem uma estrutura definida pelo Manual de Perícia Oficial em Saúde do Servidor Público Federal[1], contemplando diferentes tipos de análises pericias, a depender do tempo de afastamento concedido no atestado médico e da natureza do afastamento. O fluxo de homologação inicia-se com o envio do atestado pelo servidor através do aplicativo SOUGOV, seguido da análise preliminar realizada por um servidor da unidade SIASS. Após essa análise, o documento é encaminhado para a triagem e, caso necessário, o servidor é agendado para atendimento presencial. Os tipos de avaliação podem ser classificados em quatro categorias: perícia singular, junta médica, perícia documental e perícia odontológica. A perícia singular é conduzida por um único médico e requer a presença do servidor, sendo utilizada para afastamentos de 61 a 120 dias, no entando caso o atestado não tenha CID ou falte alguma outra informação importante para a homologação, será sempre singular mesmo que tenha menos de 60 dias de afastamento acumulados. A junta médica ocorre quando envolve dois médicos, sendo obrigatória para afastamentos superiores a 120 dias. Já a perícia documental pode ser apli-

cada a afastamentos de 15 a 60 dias, sendo realizada sem a presença do servidor, desde que o atestado não contenha CID F (transtornos mentais) ou acidente em serviço, casos em que a avaliação sempre será presencial. Por fim, a perícia odontológica segue os mesmos critérios da perícia singular, mas é realizada por um dentista. Além dessas categorias, existe a possibilidade de dispensa de perícia, que pode ocorrer quando o afastamento é de 1 a 14 dias e os documentos apresentados atendem aos critérios estabelecidos. Essa sistemática permite a correta classificação dos afastamentos, garantindo que cada caso seja tratado conforme a legislação vigente e proporcionando maior eficiência na gestão da saúde dos servidores.

Nesse cenário, o uso de tecnologias modernas, como Python[4] e Streamlit[5], emerge como uma solução eficiente para transformar dados brutos em informações acionáveis. Python, com seu amplo ecossistema de bibliotecas para análise de dados, combinado com Streamlit, uma ferramenta para criação de aplicativos interativos, possibilita o desenvolvimento de sistemas robustos para visualização e análise de dados em tempo real. Essas tecnologias permitem a criação de interfaces intuitivas que não apenas apresentam gráficos dinâmicos, mas também fornecem comparações entre períodos e métricas específicas que atendem às necessidades do SIASS.

Este trabalho tem como objetivo o desenvolvimento de um sistema interativo para uso interno dos servidores do SIASS na UFRN. O sistema possibilita a geração de gráficos em tempo real com base nos atendimentos realizados no DAS, permitindo análises quantitativas dos tipos de perícias realizadas, comparações entre períodos e visualização das proporções de atendimentos. A contribuição deste projeto está em facilitar o acompanhamento das operações realizadas pelo SIASS, fornecendo informações que podem melhorar a alocação de recursos, identificar padrões e otimizar os processos de homologação de atestados.

A relevância deste projeto reside em sua contribuição para a eficiência e transparência dos processos internos do SIASS. Ao fornecer uma plataforma intuitiva e baseada em dados, o sistema visa melhorar a gestão dos atendimentos, otimizar a alocação de recursos humanos e técnicos, e reduzir o tempo de análise dos casos. Essa integração entre tecnologia e gestão reforça o papel do SIASS como um instrumento de apoio à saúde do servidor público.

Para estruturar o desenvolvimento deste trabalho, o capítulo 2 apresenta conceitos essenciais para a compreensão do projeto, como a aplicação de ferramentas tecnológicas para análise de dados, falando sobre as bibliotecas utilizadas, como Streamlit, st_pages, Pandas, Matplotlib, Plotly, Datetime e Pygsheets, além de conceitos sobre Google Cloud, Apps Script do Google e o Google Sheets. No capítulo 3, são descritos os processos adotados na construção do sistema, desde a integração com o Google Sheets para aquisição dos dados até o desenvolvimento da interface interativa utilizando Streamlit, além da explicação sobre as funções implementadas para filtragem, tratamento e visualização das informações. O capítulo 4 apresenta a validação do sistema, demonstrando seu funcionamento e a eficiência na geração de gráficos em tempo real. Por fim, o capítulo 5 apresenta uma síntese dos principais resultados obtidos com a implementação do sistema, destacando sua contribuição para a eficiência e transparência na gestão dos atendimentos do SIASS da UFRN. Além de avaliar os impactos positivos da solução na otimização dos

processos administrativos e na visualização dinâmica dos dados, esse capítulo também propõe possíveis aprimoramentos, como a implementação de novas métricas de análise estatística, a integração com outros sistemas administrativos da universidade e a inclusão de funcionalidades avançadas, como relatórios automáticos e alertas para padrões atípicos nos atendimentos.

Capítulo 2

Fundamentação Teórica

O desenvolvimento do sistema utilizou diversas bibliotecas e tecnologias que desempenham papéis específicos na construção de um ambiente interativo, eficiente e dinâmico para os servidores do SIASS da UFRN. Cada uma dessas ferramentas foi escolhida com base em sua capacidade de atender às demandas técnicas do projeto, garantindo a integração e o processamento de dados em tempo real. A seguir, são descritas as bibliotecas e tecnologias utilizadas.

2.1 Pandas

O Pandas é uma biblioteca Python amplamente utilizada para manipulação e análise de dados. Ela permite a criação e manipulação de estruturas de dados como DataFrames e Series, que facilitam a organização, armazenamento e processamento de grandes volumes de informações de forma eficiente. Suas funcionalidades incluem filtragem, agrupamento, preenchimento de valores ausentes e operações matemáticas, tornando-a uma ferramenta indispensável para cientistas de dados e desenvolvedores.

2.2 Pygsheets

O Pygsheets é uma biblioteca Python que oferece integração com Google Sheets, permitindo acessar, modificar e atualizar planilhas em tempo real, ou seja, é uma interface simplificada para a API do Google Sheets.

Ele permite manipular dados em planilhas diretamente a partir de scripts Python, incluindo leitura, escrita, criação de novas planilhas e edição de células. Sua integração com o Google Sheets é particularmente útil para automação de tarefas e sistemas que requerem dados dinâmicos e colaborativos.

2.3 Datetime

O módulo Datetime do Python é uma biblioteca padrão projetada para manipulação de datas e horários. Ele fornece uma ampla gama de funcionalidades, como captura da data e hora atual, cálculos com intervalos de tempo, e conversões de strings para objetos datetime. Essas operações são essenciais para aplicações que dependem de filtros temporais ou cálculos baseados em tempo.

2.4 Plotly

O Plotly é uma biblioteca para criação de gráficos interativos e altamente customizáveis.

Ela suporta gráficos de linhas, barras, dispersão, pizza, mapas e até visualizações em 3D. O Plotly é conhecido por sua interatividade, permitindo que os usuários explorem dados diretamente nos gráficos, com funcionalidades como zoom, tooltips e legendas dinâmicas.

2.5 Matplotlib

O Matplotlib é uma das bibliotecas mais populares para geração de gráficos em Python. Embora menos interativa que o Plotly, é eficiente para gráficos estáticos e rápidos.

Ela suporta uma ampla gama de visualizações, como gráficos de linha, barras, dispersão, histogramas e pizza, além de oferecer personalizações detalhadas para elementos gráficos, como cores, estilos de linha e legendas.

2.6 Streamlit

O Streamlit é uma biblioteca Python projetada para simplificar o desenvolvimento de aplicações web interativas voltadas para ciência de dados, visualização de informações e prototipagem rápida. A principal característica do Streamlit é a sua facilidade de uso: com apenas algumas linhas de código, é possível criar interfaces completas com gráficos, tabelas, filtros e outros componentes interativos. Ele elimina a necessidade de configurações complexas de front-end, permitindo que os desenvolvedores se concentrem exclusivamente na lógica e no processamento de dados.

2.7 st_pages

O módulo st_pages é uma extensão do Streamlit que facilita a personalização de páginas em aplicações interativas e juntos podem deixar a navegação entre páginas mais organizada e visualmente melhor.

Ele permite estruturar e organizar aplicações em módulos separados, com títulos personalizados, ícones e menus de navegação. Essa abordagem modular torna o código mais organizado e escalável, especialmente em projetos complexos.

O st_pages é ideal para aplicações que requerem múltiplas seções, como dashboards e sistemas administrativos. Ele elimina a complexidade de configurar manualmente a navegação entre arquivos, tornando o fluxo do projeto mais eficiente e intuitivo.

2.8 Classes em python

Classes em Python são estruturas que permitem agrupar dados e comportamentos relacionados em um único objeto. Elas são a base da programação orientada a objetos (POO) e permitem criar modelos que representam entidades do mundo real. Uma classe é

definida usando a palavra-chave class e pode conter atributos e métodos que definem seu comportamento.

2.9 Apps Script

O Apps Script do Google é uma plataforma baseada em JavaScript que permite automatizar e estender as funcionalidades de serviços do Google, como Google Sheets, Docs, Drive e Gmail. Ele é uma ferramenta poderosa para criar soluções personalizadas que integram os serviços da suíte Google Workspace.

Por exemplo, com o Apps Script, é possível criar funções personalizadas em planilhas, automatizar a manipulação de dados, enviar e-mails em massa ou agendar tarefas recorrentes. Um exemplo de uso prático seria automatizar a extração de dados de uma planilha, como foi feito no projeto.

2.10 Google Cloud

O Google Cloud é uma plataforma abrangente que fornece uma ampla gama de serviços para computação em nuvem, armazenamento, análise de dados, inteligência artificial e muito mais. Essa plataforma também pode ser utilizada para configurar a autenticação de acesso ao Google Sheets, por meio da criação de uma conta de serviço e geração de uma chave de acesso JSON.

A conta de serviço criada no Google Cloud permite que aplicações externas, como scripts Python, acessem recursos protegidos, como planilhas compartilhadas, de forma segura. Este processo é fundamental para automação e integração com a API do Google Sheets. A chave de acesso JSON contém informações de autenticação, como identificadores da conta de serviço e chaves privadas, garantindo a comunicação segura com os serviços do Google.

Para entender como criar a conta de serviço e configurar o acesso, o tutorial disponível no Medium (Gspread: Trabalhando com o Google Sheets)[15] foi utilizado como referência prática. O guia detalha como habilitar a API, criar uma conta de serviço e associar a chave de acesso JSON, facilitando o uso do Pygsheets para manipulação de dados em planilhas.

2.11 Integração das Tecnologias

A integração das bibliotecas e tecnologias descritas foi realizada de maneira modular, garantindo a separação das responsabilidades de cada componente e facilitando a manutenção do código. O Streamlit atuou como a camada de interface, enquanto o Pandas e o Pygsheets forneceram o suporte para a manipulação e integração dos dados. Já o Plotly e o Matplotlib foram responsáveis pela visualização gráfica, apresentando os resultados de maneira clara e intuitiva para os usuários do sistema.

Capítulo 3

Metodologia/Implementação

O presente capítulo descreve os métodos e procedimentos adotados no desenvolvimento do sistema para o setor de saúde da Universidade Federal do Rio Grande do Norte (UFRN). Este sistema foi projetado para automatizar a análise e visualização de dados relacionados aos atendimentos realizados na Divisão de Atenção à Saúde do Servidor (DAS), vinculada ao Subsistema Integrado de Atenção à Saúde do Servidor (SIASS).

A ideia para esse sistema surgiu no dia-a-dia do trabalho no DAS, com a necessidade da utilização dos dados de atendimento dos servidores para além de uma visualização em forma de tabelas, que nem sempre é a melhor forma de analisar dados, ainda mais quando se trata de muitas informações como data de agendamento, data de inicio do atestado, tipo de perícia, todo tipo de informação pessoal a respeito de cada uma das pessoas atendias e até mesmo o próprio atestado enviado, dentre outras coisas.

O desenvolvimento foi inteiramente na linguagem Python. A biblioteca Streamlit em conjunto de sua extensão st_pages atuaram como o front-end. No back-end, responsáveis pela extração, manipulação e analise de dados, foram utilizadas as bibliotecas Pandas, Pygsheets e Datetime.

As principais funcionalidades do sistema incluem:

- Geração de gráficos em tempo real: o sistema gera gráficos interativos (linhas, colunas, barras, pizza e tabelas) utilizando as bibliotecas Plotly e Matplotlib;
- Análise de proporções: exibe a distribuição percentual dos tipos de perícias realizadas;
- Comparação entre períodos: faz comparações entre as quantidades de atendimentos realizados, sendo feitas dia-a-dia e podem começar no mesmo dia inicial selecionado mas no mês anterior ou em "x"dias anteriores, com esse "x"sendo equivalente a quantidade de dias partindo da data inicial até a data final selecionadas na barra lateral;
- Download de dados: oferece a opção de baixar os dados processados em formato de gráfico ou CSV, caso seja uma tabela;
- Automatização do fluxo de dados: os dados do Google Sheets são lidos e processados automaticamente em tempo real;
- Navegação entre múltiplas páginas.

3.1 Interface

A interface do sistema foi desenvolvida utilizando a biblioteca Streamlit, que atua como o front-end da aplicação, que foi usado principalmente para exibir gráficos interativos e tabelas com botões de download. Suas funções, como *st.plotly_chart*, *st.write* e *st.download_button*, proporcionaram uma interação direta entre o usuário e os dados visualizados. Além disso, o uso de *st.columns* ajudou a organizar os elementos de seleção gerados por *st.selectbox* de forma estruturada, sendo responsáveis pelos elementos de seleção que geram as exibições dos gráficos de acordo com a combinação, como pode ser visto na figura 3.1.

```
# Define as opções disponíveis para os diferentes parâmetros do sistema
options1 = ["Proporção geral", "Junta", "Singular", "Documental", "Odontológica"]
options2 = ["Atendimento"]
options3 = ["Linha", "Coluna", "Barra", "Pizza", "Tabela para download"]
options4 = ["Nenhum", f"{'x'} dias anteriores", "Período anterior"]

# Cria quatro colunas na interface do Streamlit
# Isso permite organizar os widgets de forma alinhada horizontalmente
col1, col2, col3, col4 = st.columns(4)

# Adiciona um seletor dropdown para cada coluna, permitindo ao usuário escolher os parâmetros
with col1:
    box_tipoDePericia = st.selectbox("Tipo de perícia:", options1)
with col2:
    box_analise = st.selectbox("Análise de:", options2)
with col3:
    box_tipoDeGrafico = st.selectbox("Tipo de Gráfico:", options3)
with col4:
    box_comparacao = st.selectbox("Comparar com:", options4)
```

Figura 3.1: Trecho do algoritmo para a selectbox do Streamlit.

Outro elemento estrutural importante para a geração dos gráficos são *st.sidebar* e *date_input*. *st.sidebar* exibe uma barra lateral onde é possível adicionar "widgets"que podem ser passados ao método através da notação de objeto. *date_input* é passado para a barra lateral da seguinte desta forma *st.sidebardate_input()*, criando inputs de data que serão utilizados para, junto dos elementos de *st.selectbox*, gerar os gráficos no período e combinação selecionada. Será abordado mais sobre esse assunto no tópico sobre as funções.

Na barra lateral é também onde foram integradas as multiplas páginas para navegação usando. A figura 3.2 mostra como Streamlit e o st_pages trabalharam juntos para facilita a navegação entre seções distintas, como "Agenda"e "Triagem". Cada página foi configurada com títulos e ícones personalizados para melhorar a experiência do usuário. Além disso, a função *add_page_title* foi usada para ajustar automaticamente o título da página ativa, *st.navigation()* para criar um menu interativo através de um dicionário recebido e *run()* para executar a página selecionada.

```python
# Estrutura de navegação do sistema
# O dicionário "pages" organiza as categorias e páginas do sistema
pages = {
    "":
    [st.Page("paginas/inicio.py", title="Inicio", icon="🏠", default=True),],
    "📅 Agenda": [
        st.Page("paginas/agenda.py", title="Geral", icon="🧍‍♀️"),
        st.Page("paginas/agenda_UFRN.py", title="UFRN", icon="🐬"),
    ],
    "💲 Triagem": [
        st.Page("paginas/Triagem.py", title="Geral",icon="🧍‍♀️"),
        st.Page("paginas/Triagem_UFRN.py", title="UFRN", icon="🐬"),
    ],
}

# Criação do menu de navegação
# O método st.navigation utiliza a estrutura definida no dicionário "pages" para criar um menu interativo
pg = st.navigation(pages)

# Adiciona o título personalizado para a página ativa
# Utiliza o título definido em cada página do dicionário "pages"
add_page_title(pg)
💡
# Executa a página selecionada pelo usuário
# O método run() carrega e exibe a lógica da página correspondente no menu
pg.run()
```

Figura 3.2: Trecho do algoritmo para a criação da navegação entre as páginas.

3.2 Aquisição dos dados

Os dados utilizados no sistema são armazenados em duas planilhas do Google Sheets, ambas atualizadas em tempo real, sendo a da triagem incrementada com novos dados a cada novo servidor que passou ou deve passar pelo atendimento e a da agenda sendo gerada por um algoritmo criado no Apps Script do Google.

Na planilha da triagem ficam os servidores que estão aguardando agendamento e que já foram atendidos. Quando um servidor é convocado para seu atendimento ou tem sua perícia documental agendada, é adicionado um evendo na agenda do Google com todos os dados referentes aquele agendamento.

Foram utilizadas essas duas fontes de dados pois na triagem tem apenas as datas de envio e de inicio do atestado. Já na agenda têm a data de atendimento, que é a própria data do evento. Desta forma é possível obter a quantidade de atestados recebido em determinado periodo e também a quantidade de atendimentos realizados. Dito isso, é importante deixar claro que não cabe aqui realizar um estudo sobre o intervalo entre a data de envio e a data de agendamento, uma vez que os atendimentos dependem de algumas variáveis, como demanda do setor no momento e disponibilidade dos peritos.

3.2.1 Apps Script

Para associar um algoritimo do Apps Script a uma planilha, basta acessar o arquivo que deseja manipular e ir no menu "Extensões > Apps Script".

O algoritmo desenvolvido, visto na figura 3.3, pode ser executado automáticamente, para isso é necessário a criação de um "gatilho". Essa funcionalidade pode ser ativada acesando o Apps Script, na mesma página onde está seu algorítimo, e indo no menu "acionadores"que fica na barra lateral. Após esse procedimento, só é necessário clicar em

"Adicionar acionador", que fica no canto inferior direito da tela e definir o que e qual o intervalo de acionamento desse gatilho de acordo com a necessidade de atualização da planilha.

```
function importarEventosMultiplasAgendas() {
    // Obtenha a planilha ativa
    var planilha = SpreadsheetApp.getActiveSpreadsheet().getActiveSheet();

    // Lista de IDs de agendas compartilhadas (adicione os IDs necessários)
    var idsAgendas = [
        'c_2d5e42a253263fd6e35c98329dab9b7d489bbf12c6caaa0d652f09bb8335c9190group.calendar.google.com', //ARILDO
        'c_043e5ea6ca2fd147b549839807e23f650846e9da246f837179416f7f3af63621 0group.calendar.google.com', //CARLOS
        'c_98f50b8638084e7532d17827f3aa41963066ba36f3c046ef573608bfd8e04794 0group.calendar.google.com', //INGRID
        'c_e1ebcee5e7e1c20728891da05cfac78f7a85ecb8cf4b9cd83ba7f2537356b1c80group.calendar.google.com', //JUNTA
        'c_75431b58aba46a9e7609f8d6e46c595157e9a672137e98d3cbebcbbda7909b7c0group.calendar.google.com', //JUNTA ODONTO
        'c_a01e5bde9e48308fae8480741de224b6bd63cb5489b5c57c71530c28515cb4ce0group.calendar.google.com', //MANÇOS
        'c_c75a75f85840ec24c330986663392a34f80df0c467c89fcf67ca5e9f8bfb77a40group.calendar.google.com', //MARA
        'c_92b8fe783d1dc0c5b841046f366d176c17d3b1fd9d929ae32c96afb41e6fe1170group.calendar.google.com', //MARILUCIA
        'c_c5a202b33acfb13e1589b09e8f3f34d95162915c7ad9c414f6d1129c771b02de0group.calendar.google.com', //ODONTO ANDRE
        'c_f070b216002121a1c160a112f1ee39e0bb036a8a66afd515eb15b1c4ffa0fda20group.calendar.google.com', //ODONTO ANDRE
        'c_d0df7303b5d41ba7ba397ccafadfe578cc8c03883fb4e13b5b16be12195627250group.calendar.google.com', //ODONTO NIVIA
        'c_c5cb8537955bc41bef80bef17b70c414eac3b80798a3f6f2016266052ba0e24c0group.calendar.google.com', //ODONTO RICARDO
        'c_b456101ce6d7f25088a4934ac33e162f3460fa992b43e8e3b38b5f350ba8e15e0group.calendar.google.com', //ODONTO SERGIO
        'c_4710d0d5ccb5f02ad41e716b81401cb51fa9497b9e3f2300b1e4323dc95f6b4a0group.calendar.google.com', //PERICIA EM SAUDE
        'c_17c8e83c6ce90171bb8dc87116d14878f99baaec65966405e69db315702e5402 0group.calendar.google.com', //REMARCADO
        'c_e472fcd974abf86e27faef972f7498f07742c4e28f5eb3657ba5714acc77a8ba0group.calendar.google.com', //RENAN
        'c_9ec624dd11ca4b01665a9c5c8566f26ed75d1b14ff7845c42d53146fd90bad670group.calendar.google.com', //TABATA'
        'c_5d68a2979db96d1f9f8d76e01aa077b6d33bff5bbd5ca4fa43e2677daa9cb00e0group.calendar.google.com', //THIAGO
        'c_23c0c6f6eed390eedc81662a9316ed5359af32d8e700f83bedad4527113410 8e0group.calendar.google.com' //VINICIUS
    ];

    // Intervalo de datas para buscar os eventos
    var dataInicio = new Date("2020-10-23"); //primeira data com eventos de perícia na agenda
    var dataFim = new Date(); //para pegar a data atual

    // Limpar dados existentes na planilha
    planilha.clear();

    // Escrever cabeçalho
    planilha.getRange("A1:G1").setValues([["Título", "Data Início", "Data Fim", "DESCRIÇÃO", "LOCAL", "DATA CRIAÇÃO", "TAG"]]);

    // Armazena todos os eventos combinados
    var dados = [];

    // Iterar por cada ID de agenda
    for (var i = 0; i < idsAgendas.length; i++) {
        var calendario = CalendarApp.getCalendarById(idsAgendas[i]);
        if (!calendario) {
            Logger.log('Calendário não encontrado para o ID: ' + idsAgendas[i]);
            continue; // Pula este calendário se não for encontrado
        }
        var eventos = calendario.getEvents(dataInicio, dataFim);

        // Extrair dados dos eventos e adicioná-los ao array 'dados'
        for (var j = 0; j < eventos.length; j++) {
            var evento = eventos[j];
            dados.push([
                evento.getTitle(),
                evento.getStartTime(),
                evento.getEndTime(),
                evento.getDescription(),
                evento.getLocation(),
                evento.getDateCreated(),
                calendario.getName() // Nome da agenda (exemplo: JUNTA MÉDICA)
            ]);
        }
    }

    // Ordenar os dados por Data Início (coluna 2)
    dados.sort(function(a, b) {
        return new Date(a[1]) - new Date(b[1]); // Ordena pela segunda coluna (Data Início)
    });

    // Verificar se há eventos para escrever na planilha
    if (dados.length > 0) {
        planilha.getRange(2, 1, dados.length, dados[0].length).setValues(dados);
    } else {
        planilha.getRange("A2").setValue("Nenhum evento encontrado no intervalo de datas especificado.");
    }
}
```

Figura 3.3: Algoritmo do Apps Script responsável pela coleta de dados da agenda e armazenamento em uma planilha do Google.

3.2.2 Google Cloud

Para que se tenha uma conexão em tempo real do algorito com as planilhas é preciso criar uma conta de serviço e uma chave de acesso no google cloud, para entender melhor o processo foi utilizado o tutorial disponível no Medium (Gspread: Trabalhando com o Google Sheets)[15].

O processo pode ser dividido nas seguintes etapas:

- Abrir o Google Cloud logado no seu gmail e acessar o menu "Console";

- Criar um projeto no Google Cloud e habilitar a API do Google Sheets;
- Acessar a barra lateral e ir em "Contas de serviço";
- Criar uma nova conta de serviço, fornecendo um nome e uma descrição;
- Selecionar a conta de serviço criada e ir para a aba "Chaves"e clicar em "Adicionar chave";
- Selecionar o formato JSON, confirmar a criação e fazer o download da chave para o projeto.

A conta de serviço é criada como um tipo de email e após o processo descrito acima só é necessário acessar a planilha e compartilhá-la com ela.

3.2.3 Pygsheets e Pandas

A interação com a planilha foi realizada por meio da biblioteca Pygsheets, que permite acessar, ler e escrever dados diretamente a partir de scripts Python. Essas planilhas funcionaram como o banco de dados do sistema, centralizando as informações necessárias para as análises e visualizações. A biblioteca Pandas foi usada para transformar os dados obtidos em um Dataframe.

```
# Autenticação com o Google Sheets
# 'pygsheets.authorize' realiza a autenticação usando um arquivo de chave de serviço fornecido pelo google cloud no formato JSON
credenciais = pygsheets.authorize(service_file='contaDeServico.json')

# PLANILHA DA TRIAGEM
# URL da planilha Google da triagem
meuArquivoSheets = "https://docs.google.com/spreadsheets/d/"

# Abre a planilha pelo URL utilizando as credenciais fornecidas
arquivo = credenciais.open_by_url(meuArquivoSheets)

# Acessa a aba chamada "Triagem" dentro da planilha
aba = arquivo.worksheet_by_title("Triagem")

# Obtém todos os valores da aba "Triagem"
# Retorna os dados como uma lista de listas, onde cada sublista representa uma linha da planilha
data = aba.get_all_values()

# Cria um DataFrame com os dados obtidos
# A primeira linha da planilha é usada como nomes das colunas
df = pd.DataFrame(data, columns=data[0]).drop(0)#drop para remover a linha de cabeçalho que estava duplicada
```

Figura 3.4: Exemplo de algoritmo para receber dados de planilhas em tempo real.

Ainda no mesmo arquivo que está o trecho da figura 3.4, que mostra como os dados são acessados, tem algumas manipulações de dados, para garantir que os Dataframes não tenham cabeçalho duplicado, colunas duplicadas, linhas vazias e que as datas fiquem no formato correto para comparações futuras como é visto na figura 3.5.

```python
# Checar se a primeira linha é igual ao cabeçalho
if data[0] == data[1]:  # Confirma duplicação
    df = pd.DataFrame(data[1:], columns=data[0])  # Usar as linhas a partir da segunda
else:
    df = pd.DataFrame(data[1:], columns=data[0])

# Remove linhas onde as colunas 'TIPO PERICIA' e 'Situação' têm valores ausentes (NaN)
df_sem_linhas_vazias2 = df.dropna(subset=['TIPO PERICIA','Situação'])

# Remove colunas duplicadas no DataFrame
# A lógica loc[:, ~df.columns.duplicated()] seleciona todas as colunas onde a duplicidade é False. "~" é uma negativa na operação.
df_sem_linhas_vazias2 = df_sem_linhas_vazias2.loc[:, ~df_sem_linhas_vazias2.columns.duplicated()]

# Converte a coluna 'DATA DO ATESTADO' para o tipo datetime no formato especificado ("%d/%m/%Y")
# Caso encontre valores inválidos ou não conversíveis, esses são convertidos para NaT (Not a Time)
df_sem_linhas_vazias2['DATA DO ATESTADO'] = pd.to_datetime(
    df_sem_linhas_vazias2['DATA DO ATESTADO'], # Coluna que contém as datas como strings
    format="%d/%m/%Y", # Define o formato esperado das datas (dia/mês/ano)
    errors='coerce'  # Converte valores inválidos para NaT
).dt.date # Extrai apenas a parte de data (ano, mês e dia) sem informações de horário
```

Figura 3.5: Tratamento de dados inicial do DataFrame com as planilhas.

3.3 Funções

3.3.1 filtroDeDatas

A função filtroDeDatas (figura 3.6) tem como objetivo filtrar um conjunto de dados com base em um intervalo de datas específico. Para isso, ela recebe um DataFrame contendo informações, além de duas datas que representam os limites do período desejado, start_date e end_date. Seu funcionamento consiste em criar uma cópia do DataFrame original, garantindo que os dados não sejam alterados diretamente, e, em seguida, aplicar um filtro que seleciona apenas as linhas que tiverem a data de inicio do atestado ou a data de atendimento, respectivamente para os dados da triagem e agenda, está dentro do intervalo fornecido.

O processo de filtragem ocorre através do uso da função .loc[], um método eficiente do Pandas para seleção de dados. A condição aplicada verifica se cada linha da coluna com as datas contém uma data maior ou igual à start_date e menor ou igual à end_date. Apenas as linhas que atendem a essa condição são mantidas no novo DataFrame. Ao final, o DataFrame filtrado é retornado, contendo apenas os registros que se enquadram no período especificado.

```python
#FILTRO DE DATAS
def filtroDeDatas(df_sem_linhas_vazias2, start_date, end_date):
    """
    Filtra o DataFrame fornecido com base em um intervalo de datas.

    Args:
        df_sem_linhas_vazias2 (pd.DataFrame): DataFrame original que contém a coluna 'DATA DO ATESTADO'.
        start_date (datetime.date): Data inicial para o filtro.
        end_date (datetime.date): Data final para o filtro.

    Retorna:
        pd.DataFrame: DataFrame contendo apenas as linhas cujo valor em 'DATA DO ATESTADO' está dentro do intervalo especificado.
    """

    # Cria uma cópia do DataFrame original para evitar alterações acidentais nos dados originais
    copia_df_semLinhasVazias = df_sem_linhas_vazias2.copy()

    # Filtra as linhas do DataFrame com base no intervalo de datas fornecido
    # O filtro seleciona apenas as linhas onde 'DATA DO ATESTADO' está entre start_date e end_date
    df_filtrado = copia_df_semLinhasVazias.loc[(copia_df_semLinhasVazias['DATA DO ATESTADO'] >= start_date) & (copia_df_semLinhasVazias
    ['DATA DO ATESTADO'] <= end_date)]

    # Retorna o DataFrame filtrado
    return df_filtrado
```

Figura 3.6: Função de filtro de datas.

3.3.2 datainput

A função dataInput_session (figura 3.7) é responsável por gerenciar a entrada de datas na aplicação, garantindo que as seleções do usuário sejam armazenadas e mantidas ao longo da navegação. Para isso, ela utiliza o st.session_state (figura 3.8), um recurso do Streamlit que permite preservar valores entre diferentes interações dentro da mesma sessão. Inicialmente, a função verifica se as datas de início e término já estão armazenadas no estado da sessão. Caso não estejam, ela define valores padrão, sendo a data de início configurada como 1º de novembro de 2024 e a data de término definida automaticamente como a data atual do sistema. Essa inicialização evita que as datas fiquem vazias quando o usuário acessa a aplicação pela primeira vez.

Após definir ou recuperar as datas do estado da sessão, a função exibe dois inputs interativos na barra lateral da interface usando st.sidebar.date_input. Esses inputs permitem que o usuário selecione um período de análise, garantindo que apenas datas dentro de um intervalo válido possam ser escolhidas. O menor valor permitido é 18 de março de 2020, e o máximo sempre será o dia atual, evitando que o usuário selecione datas futuras. O formato utilizado para exibição das datas é "YYYY-MM-DD", garantindo consistência com outras partes do sistema que realizam filtragens e comparações temporais.

No final da função, um trecho de código verifica se o usuário alterou alguma das datas. Caso o valor selecionado para data_inicio ou data_fim seja diferente do que está salvo no st.session_state (figura 3.8), o novo valor é atualizado no estado da sessão. Esse mecanismo evita que as escolhas do usuário sejam perdidas ao navegar entre páginas da aplicação. Com isso, mesmo que o usuário saia da página e retorne posteriormente, as datas que ele selecionou permanecerão as mesmas. Esse comportamento melhora significativamente a experiência do usuário, garantindo que ele não precise redefinir os filtros de tempo a cada nova interação. Após todas essas operações, a função retorna as datas selecionadas pelo usuário, permitindo que outras partes do sistema utilizem essas informações para filtrar e processar os dados exibidos.

```python
def dataInput_session():

    """
    Função responsável por gerenciar os inputs de data e manter o estado persistente na sessão do Streamlit.
    Utiliza o `st.session_state` para armazenar as datas de início e término, garantindo que as seleções do usuário
    não sejam perdidas ao navegar entre páginas ou realizar outras interações no sistema.

    Retorna:
        - data_inicio (datetime): Data inicial selecionada pelo usuário.
        - data_fim (datetime): Data final selecionada pelo usuário.
    """

    # Verifique e defina o estado inicial das datas se necessário
    if 'data_inicio' not in st.session_state: # Verifica se a chave 'data_inicio' está no estado da sessão. Se não estiver, a chave é
    adicionada com a data inicial 2023-01-01.
        st.session_state['data_inicio'] = pd.to_datetime('2024-11-01')

    if 'data_fim' not in st.session_state: #Faz o mesmo para a chave 'data_fim', definindo a data final como 2024-06-06.
        st.session_state['data_fim'] = pd.Timestamp.now().normalize()

    # Inputs de data na barra lateral
    data_inicio = st.sidebar.date_input(
        "Data de início", # Texto do rótulo do input
        value=st.session_state['data_inicio'], # Data inicial - valor padrão da data de início
        min_value=pd.to_datetime('2020-03-18'), # Data mínima permitida
        max_value=pd.Timestamp.now().normalize(), # Data máxima permitida (sempre o dia atual)
        format="YYYY-MM-DD", # Formato da data exibida - escolhido dessa forma para a melhor comparação entre as data na função de filtro
        de datas.
        # help="Data mínima provisória. Será alterada de acordo com desenvolvimento do sistema."
    )

    data_fim = st.sidebar.date_input(
        "Data de término",
        value=st.session_state['data_fim'], # Data final - valor padrão da data final
        min_value=pd.to_datetime('2020-03-18'),
        max_value=pd.Timestamp.now().normalize(),
        format="YYYY-MM-DD",
        # help="Data máxima provisória. Será alterada de acordo com desenvolvimento do sistema."
    )
```

Figura 3.7: Primeira parte da função para receber e preservar as entradas de datas.

```python
    """st.session_state

    É um objeto que permite armazenar e compartilhar dados entre diferentes execuções do código em uma sessão do usuário. Ele atua como
    uma memória persistente que mantém o estado entre as interações do usuário dentro da mesma sessão do navegador.

    'st.session_state' permite armazenar dados que devem persistir entre diferentes execuções do script dentro da mesma sessão do
    usuário. Por exemplo, se um usuário seleciona uma data em uma página e navega para outra página, o valor da data selecionada pode ser
    mantido usando 'st.session_state'.

    https://docs.streamlit.io/develop/api-reference/caching-and-state/st.session_state

    """
    # Atualizar o estado se as datas mudarem
    # Serve para a data selecionada na barra lateral não ser resetada quando navegar entre as páginas
    if data_inicio != st.session_state['data_inicio']: #Verifica se a data de início selecionada pelo usuário é diferente da armazenada
    no estado da sessão.
        st.session_state['data_inicio'] = data_inicio #Se a data for diferente, atualiza o estado da sessão com a nova data.

    if data_fim != st.session_state['data_fim']: #Faz o mesmo para a data de término.
        st.session_state['data_fim'] = data_fim

    # Retorna as datas selecionadas pelo usuário
    return data_inicio,data_fim
```

Figura 3.8: Segunda parte da função para receber e preservar as entradas de datas.

3.3.3 preencher_datas_faltantes

A função preencher_datas_faltantes (figura 3.9) tem como objetivo garantir que todas as datas dentro de um intervalo especificado estejam presentes no DataFrame, preenchendo automaticamente os dias ausentes com um valor padrão de 0.

O funcionamento da função inicia com a criação de uma série de datas contínuas, utilizando a função pd.date_range(), que gera uma sequência de datas entre data_inicio e

data_fim. Em seguida, um novo DataFrame (df_todas_as_datas) é criado contendo essa sequência de datas na coluna definida pelo parâmetro data_col. O próximo passo é garantir que a coluna de datas do DataFrame original esteja no formato adequado (datetime64[ns]), convertendo-a com pd.to_datetime(). Isso assegura a consistência na operação de junção que será realizada a seguir.

A etapa mais importante é a junção dos dois DataFrames, utilizando a função pd.merge(). Essa operação é feita com a opção how='left', garantindo que todas as datas do DataFrame recém-criado sejam mantidas, enquanto os valores do DataFrame original são combinados onde houver correspondência de datas. Se um dia não existia no conjunto original, ele será incluído, mas com um valor NaN na coluna de valores (valor_col).

Para corrigir esses valores ausentes, a função fillna(0) é utilizada, preenchendo os espaços vazios com 0. Além disso, a conversão para int (astype(int)) assegura que a coluna de valores mantenha um formato adequado para análise numérica, evitando inconsistências na manipulação futura dos dados. Por fim, a coluna de datas é convertida novamente para o formato datetime.date, garantindo compatibilidade com outras funções que podem utilizar esse DataFrame.

O retorno da função é um DataFrame completo e estruturado, onde todas as datas do intervalo especificado estão presentes, sem lacunas e com valores devidamente preenchidos.

Na figura 3.9 é possível ver o algoritmo implememtado:

```python
def preencher_datas_faltantes(df, data_inicio, data_fim, data_col, valor_col):
    """
    Preenche datas faltantes em um DataFrame com valor 0 e retorna um DataFrame contendo todas as datas
    do intervalo especificado, com as colunas de data no formato datetime.date.

    Args:
        df (pd.DataFrame): DataFrame original contendo as colunas de data e valor.
        data_inicio (str): Data de início do período no formato 'YYYY-MM-DD'.
        data_fim (str): Data de fim do período no formato 'YYYY-MM-DD'.
        data_col (str): Nome da coluna que contém as datas no DataFrame original.
        valor_col (str): Nome da coluna que contém os valores correspondentes às datas.

    Retorna:
        pd.DataFrame: DataFrame com todas as datas preenchidas e valores ausentes definidos como 0.
    """
    # Gera uma série contínua de datas no intervalo selecionado
    todas_as_datas = pd.date_range(start=data_inicio, end=data_fim)

    # Cria um DataFrame com todas as datas
    df_todas_as_datas = pd.DataFrame({data_col: todas_as_datas})

    # Converte a coluna de data no DataFrame original para datetime64[ns]
    # Isso garante que as operações de mesclagem sejam consistentes
    df[data_col] = pd.to_datetime(df[data_col])

    # Realiza a mesclagem do DataFrame original com o DataFrame de todas as datas
    # O método 'left' adiciona as linhas ausentes preenchendo-as com valores NaN
    df_completo = pd.merge(df_todas_as_datas, df, on=data_col, how='left')

    # Preenche valores ausentes na coluna de valores com 0 e converte para inteiro
    df_completo[valor_col] = df_completo[valor_col].fillna(0).astype(int)

    # Converte as datas para o formato datetime.date para compatibilidade futura
    df_completo[data_col] = df_completo[data_col].dt.date

    return df_completo
```

Figura 3.9: Função para preencher as datas que não houveram atendimento.

3.3.4 get_dataNoMesAnterior

A função na figura 3.10 tem como subtrair um mês da data fornecida pelo usuário e retornar essa nova data.

O funcionamento da função começa verificando se a dataSlecionada é do tipo string. Caso seja, a função a converte para um objeto do tipo datetime.date utilizando datetime.strptime(), garantindo que a data esteja no formato correto para ser manipulada.

Em seguida, a função determina o mês e o ano anteriores à data selecionada. Isso é feito reduzindo um mês da data original (dataSlecionada.month - 1). Caso o mês seja janeiro, o novo mês será dezembro do ano anterior, garantindo a transição correta entre anos.

Após isso, a função tenta criar uma nova data utilizando replace(), onde apenas os valores do ano e do mês são alterados. No entanto, pode haver um problema caso a data original seja um dia que não exista no mês anterior, como 31 de março, que não tem equivalente direto em fevereiro. Para lidar com isso, a função captura a exceção ValueError e resolve o problema ajustando a data para o último dia do mês anterior. Isso é feito definindo o dia como 1 e subtraindo um dia (timedelta(days=-1)), garantindo que datas como 31 de março se tornem 28 ou 29 de fevereiro, dependendo do ano.

A função retorna a nova data corretamente ajustada para o mês anterior, garantindo precisão mesmo em cenários de meses com diferentes quantidades de dias.

```python
def get_dataNoMesAnterior(dataSlecionada):
    """
    Retorna a mesma data no mês anterior.

    Args:
        dataSlecionada (datetime): A data selecionada pelo usuário.

    Returns:
        new_date (datetime): A nova data no mês anterior.
    """
    # Converte para datetime, caso necessário
    if isinstance(dataSlecionada, str):
        dataSlecionada = datetime.strptime(dataSlecionada, '%Y-%m-%d').date()

    # Subtrai um mês da data
    month = dataSlecionada.month - 1 if dataSlecionada.month > 1 else 12
    year = dataSlecionada.year if dataSlecionada.month > 1 else dataSlecionada.year - 1

    # Tenta criar a nova data no mês anterior
    try:
        new_date = dataSlecionada.replace(year=year, month=month)
    except ValueError:
        # Caso a data do mês anterior não exista (ex: 31/03 -> 28/02 ou 29/02 em anos bissextos)
        new_date = dataSlecionada.replace(year=year, month=month, day=1) + timedelta(days=-1)

    return new_date
```

Figura 3.10: Função para subtrair um mês da data recebida.

3.4 Classes

A geração de gráficos é uma das principais funcionalidades do sistema, permitindo que os usuários interpretem os dados de forma visual e interativa. Essa tarefa foi dividida em dois módulos principais: *graficos* e *multGraficos*.

Suas implementações permitem a criação de diversas visualizações de dados, proporcionando uma análise detalhada e intuitiva. Essa abordagem orientada a objetos facilita

a reutilização e manutenção do código, além de melhorar a organização das funções que manipulam e representam visualmente os dados.

Elas possuem diversos métodos especializados na criação de diferentes tipos de gráficos, incluindo gráficos de linha, colunas verticais, barras horizontais e gráficos de pizza, que são úteis para analisar a distribuição de dados ao longo do tempo e comparar proporções. Além disso, ela conta com métodos auxiliares, como tabela_para_download, que permite aos usuários exportar os dados tratados em formato CSV, e mostrar_mensagem_nao_aplicavel, que exibe alertas para combinações de filtros e gráficos que não fazem sentido.

3.4.1 Grafico

Inicialmente pensada para ser a única classe do projeto, *Grafico* foi feita de forma que seus métodos pudessem ser passados para apenas um Dataframe por vez, como pode ser visto em seu construtor na figura 3.11.

O construtor da classe Grafico, definido pelo método especial __init__, tem a função de inicializar um objeto da classe a partir de um DataFrame do Pandas, que será utilizado para gerar gráficos e tabelas. Esse método é chamado automaticamente sempre que um novo objeto da classe Grafico é instanciado.

O funcionamento do construtor começa com a definição do parâmetro df, que deve ser um DataFrame do Pandas contendo os dados que serão manipulados. O uso da anotação de tipo df: pd.DataFrame não é obrigatório, mas ajuda a documentar o código e deixar claro que esse argumento deve ser um objeto do tipo DataFrame. Dentro do método, o parâmetro df é armazenado como um atributo da instância, acessível através de self.df. Isso significa que qualquer método dentro da classe pode acessar e utilizar esse conjunto de dados sem precisar receber o DataFrame diretamente como argumento.

A principal vantagem dessa abordagem é que a classe Grafico pode operar de maneira orientada a objetos, onde cada instância da classe pode trabalhar com diferentes conjuntos de dados sem interferência entre si. Por exemplo, se forem criados dois objetos da classe Grafico com DataFrames distintos, cada um manterá seus próprios dados e poderá gerar gráficos diferentes sem conflitos. Isso facilita a modularização do código, evitando a necessidade de passar os mesmos dados repetidamente para cada função. Além disso, ao armazenar o DataFrame dentro do objeto, o código se torna mais organizado e escalável, permitindo adicionar novos métodos sem precisar modificar a estrutura básica da classe.

```python
class Grafico:

    """
    Classe responsável por criar gráficos interativos e tabelas para download com base em um DataFrame
    fornecido.

    Métodos:
        - grafico_linha: Cria gráficos de linha.
        - grafico_coluna: Cria gráficos de colunas verticais.
        - grafico_barras_horizontal: Cria gráficos de barras horizontais.
        - tabela_para_download: Exibe e disponibiliza tabelas para download em formato CSV.
        - mostrar_mensagem_nao_aplicavel: Exibe mensagens informando combinações inválidas.
        - grafico_pizza: Cria gráficos de pizza.
        - grafico_coluna_tipoPericia: Cria gráficos de colunas para tipos de perícia utilizado na proporção
        geral.
        - grafico_barras_horizontal_tipoPericia: Cria gráficos de barras horizontais para tipos de perícia
        utilizado na proporção geral.

    Atributos:
        df (dataframe do pandas): dataframe manipulado pelo pandas com os dados a serem usados nos gráficos.
    """

    def __init__(self, df: pd.DataFrame):
        """
        Inicializa a classe com um DataFrame.

        Args:
            df (pd.DataFrame): O DataFrame contendo os dados que serão utilizados nos gráficos.
        """
        self.df = df
```

Figura 3.11: Construtor da classe *Gráfico*.

Nas figuras 3.12, 3.13, 3.14, 3.15, 3.16, 3.17, 3.18, 3.19 e 3.20 a seguir veremos os métodos implementados para a classe.

```python
    def grafico_linha(self, colunas, data_inicio, data_fim, tipoPericia, cor):

        """
        Cria e exibe um gráfico de linha dentro do periodo selecionado.

        Args:
            colunas (list): Lista com os nomes das colunas [eixo X, eixo Y].
            data_inicio (datetime): Data de início do período.
            data_fim (datetime): Data de término do período.
            tipoPericia (str): Tipo de perícia a ser exibido no título.
            cor (str): Cor da linha.
        """

        x_col, y_col = colunas
        # Cria a figura
        fig = go.Figure()

        # Adiciona a linha do período atual
        fig.add_trace(go.Scatter(
        x=self.df[x_col],
        y=self.df[y_col],
        mode='lines+markers',
        name=f'Selecionado: {data_inicio} a {data_fim}',
        text=self.df[x_col],
        hovertemplate='<br>Data: %{text}<br>Valor: %{y}',
        line=dict(color=cor)
        ))

        # Configura o layout
        fig.update_layout(
        title=f'{tipoPericia} - perícias ao longo o tempo.',
        title_x=0.1,
        title_xanchor='left',
        title_y=0.85,
        xaxis_title='Data',
        yaxis_title='Quant. de perícias',
        hovermode='x unified',
        yaxis=dict(tickformat='.'),
        legend=dict(x=1, y=1, xanchor='right', yanchor='bottom', font=dict(size=10))
        )

        st.plotly_chart(fig)
```

Figura 3.12: Método da classe *Gráfico* para o gráfico de linha.

```python
def grafico_coluna(self, colunas, data_inicio, data_fim, tipoPericia, cor):

    """
    Cria e exibe um gráfico de coluna dentro do período selecionado.

    Args:
        colunas (list): Lista com os nomes das colunas [eixo X, eixo Y].
        data_inicio (datetime): Data de início do período.
        data_fim (datetime): Data de término do período.
        tipoPericia (str): Tipo de perícia a ser exibido no título.
        cor (str): Cor da linha.
    """

    x_col, y_col = colunas
    # Cria a figura
    fig = go.Figure()

    # Adiciona as colunas do período atual
    fig.add_trace(go.Bar(
        x=self.df[x_col],
        y=self.df[y_col],
        name=f'Selecionado: {data_inicio} a {data_fim}',
        text=self.df[y_col],
        textposition='outside',
        hovertemplate='<br>Data: %{x}<br>Valor: %{y}',
        marker=dict(line=dict(color='Grey', width=2), color=cor)
    ))

    # Configura o layout
    fig.update_layout(
        title=f'{tipoPericia} - perícias ao longo o tempo.',
        title_x=0.1,
        title_xanchor='left',
        title_y=0.85,
        xaxis_title='Data',
        yaxis_title='Quant. de perícias',
        hovermode='x unified',
        yaxis=dict(tickformat='.'),
        xaxis=dict(title='Data',dtick = 1,type='category'),
        legend=dict(x=1, y=1, xanchor='right', yanchor='bottom', font=dict(size=10)),
        barmode='group'
    )

    st.plotly_chart(fig)
```

Figura 3.13: Método da classe *Gráfico* para o gráfico de coluna.

```python
def grafico_barras_horizontal(self, colunas, data_inicio, data_fim, tipoPericia, cor):

    """
    Cria e exibe um gráfico de barras dentro do periodo selecionado.

    Args:
        colunas (list): Lista com os nomes das colunas [eixo X, eixo Y].
        data_inicio (datetime): Data de início do período.
        data_fim (datetime): Data de término do período.
        tipoPericia (str): Tipo de perícia a ser exibido no título.
        cor (str): Cor da linha.
    """

    x_col, y_col = colunas
    fig = go.Figure()

    fig.add_trace(go.Bar(
        x=self.df[y_col],
        y=self.df[x_col],
        orientation='h',
        name=f'Selecionado: {data_inicio} a {data_fim}',
        text=self.df[y_col],
        textposition='auto',
        hovertemplate='<br>Data: %{y}<br>Valor: %{x}',
        marker=dict(line=dict(color='Grey', width=2), color= cor)
    ))

    fig.update_layout(
        title=f'{tipoPericia} - perícias ao longo o tempo.',
        title_x=0.1,
        title_xanchor='left',
        title_y=0.85,
        xaxis_title='Quant. de perícias',
        yaxis_title='Data',
        hovermode='y unified',
        xaxis=dict(tickformat='.'),
        yaxis=dict(title='Data',dtick = 1,type='category'),
        legend=dict(x=1, y=1, xanchor='right', yanchor='bottom', font=dict(size=10)),
        barmode='stack'
    )

    st.plotly_chart(fig)
```

Figura 3.14: Método da classe *Gráfico* para o gráfico de barra.

Foram implementaadas duas versões para a exibição das tabelas. A primeira é mais simples e vai exibir a todas as informações que estão no trecho filtrado da base de dados, a segunda oculta as informações pessoais através de um filtro aplicado com o uso de uma lista com palavras chave e do método *drop()*.

```python
def tabela_para_download(self):
    st.write(self.df)
    st.download_button(
        label="Baixar tabela como CSV",
        data=self.df.to_csv(index=False),
        file_name='dados.csv',
        mime='text/csv'
    )

def tabela_para_download(self):

    """
    Cria e exibe uma tabela para download com certos dados ocultos. Também cria um botão para fazer o
    download da mesma.

    """

    # Lista de colunas que devem ser excluídas, se existirem no DataFrame
    colunas_para_excluir = ["PRECISA DE ENFERMAGEM", "CPF", "NOME", "TELEFONE / EMAIL", "OBSERVAÇÕES",
    "NOTAS", "OCORRENCIAS", "COVID-19-BACKUP", "TIPO_EXTERNA", "PERITO","", "DESCRIÇÃO", "LOCAL",
    "Título", "DATA CRIAÇÃO"]

    # Removendo as colunas indesejadas do DataFrame, se existirem
    df_filtrado = self.df.drop(columns=[col for col in colunas_para_excluir if col in self.df.columns])

    # Exibindo o DataFrame filtrado
    st.write(df_filtrado)

    # Botão para download do DataFrame como CSV
    st.download_button(
        label="Baixar tabela como CSV",
        data=df_filtrado.to_csv(index=False),
        file_name='dados.csv',
        mime='text/csv'
    )
```

Figura 3.15: Método da classe *Gráfico* para tabelas.

A imagem abaixo é um método implementado para exibir uma mensagem informando quando a combinação escolhida for inválida para gerar uma visualização de dados.

```python
def mostrar_mensagem_nao_aplicavel(self):
    """
    Exibe uma mensagem centralizada no Streamlit informando que a combinação não é aplicável.
    """
    st.write(
        """
        <div style='display: flex; justify-content: center; align-items: center; height: 300px;'>
            <h1 style='font-size: 30px; color: #FF0000; text-align: center;'>
                Não se aplica para essa combinação. Selecione outro tipo de exibição.
            </h1>
        </div>
        """,
        unsafe_allow_html=True
    )
```

Figura 3.16: Método da classe *Gráfico* para quando não existe visualização para determinada combinação.

Gráficos feitos para "proporção geral"vão comparar todos os tipos de atendimento dentro do periodo selecionado.

```python
#PARA PROPORÇÃO GERAL
def grafico_pizza(self, colunas, cores):
    """
    Gera um gráfico de pizza com os dados fornecidos. Usado na proporção geral para mostrar a proporção
    entre os tipos de perícia.

    Parâmetros:
        colunas (list): lista com os nomes das colunas.
        cores (list): cores para cada tipo de perícia.
    """
    categoria_col, valor_col = colunas

    # Definir cores específicas para cada categoria
    colors = {
        'SINGULAR': cores[0],
        'DOCUMENTAL': cores[1],
        'JUNTA': cores[2],
        'ODONTO': cores[3]
    }

    # Criação da figura do gráfico de pizza
    fig = go.Figure()

    fig.add_trace(go.Pie(
        labels=self.df[categoria_col],
        values=self.df[valor_col],
        hoverinfo='label+value',
        textinfo='label+percent',
        textfont=dict(size=14),
        hole=0.4,  # Se quiser um gráfico de pizza tipo donut
        # marker=dict(line=dict(color='white', width=2))  # Bordas brancas para separar segmentos
        marker=dict(line=dict(color='Grey', width=2), colors=[colors.get(cat, '#7f7f7f') for cat in self.df
        [categoria_col]])  # Usa as cores definidas
    ))

    # Configuração do layout
    fig.update_layout(
        title=f'Proporção Geral dos Tipos de Perícia',
        title_x=0.5,
        title_xanchor='center',
        showlegend=True,
        legend=dict(font=dict(size=10), orientation='h', x=0.5, xanchor='center', y=-0.1)
    )

    st.plotly_chart(fig)
```

Figura 3.17: Método da classe *Gráfico* para o gráfico de pizza de proporção geral.

```python
def grafico_coluna_tipoPericia(self, colunas, cores):
    """
    Cria e exibe um gráfico de coluna para a proporção geral comparando os tipos de atendimento.

    Parâmetros:
        colunas (list): lista com os nomes das colunas [categoria, valor].
        cores (list): cores de cada pericia.
    """
    categoria_col, valor_col = colunas

    # Definir cores específicas para cada categoria
    colors = {
        'SINGULAR': cores[0],
        'DOCUMENTAL': cores[1],
        'JUNTA': cores[2],
        'ODONTO': cores[3]
    }

    # Criação da figura
    fig = go.Figure()

    # Adicionar os dados ao gráfico
    fig.add_trace(go.Bar(
        x=self.df[categoria_col],
        y=self.df[valor_col],
        text=self.df[valor_col],
        textposition='outside',
        hovertemplate='<br>Tipo de Perícia: %{x}<br>Valor: %{y}',
        marker=dict(line=dict(color='Grey', width=2), color=[colors.get(cat, '#7f7f7f') for cat in self.df
        [categoria_col]])  # Usa as cores definidas
    ))

    # Configuração do layout
    fig.update_layout(
        title=f'Proporção Geral de Tipos de Perícia',
        title_x=0.5,
        title_xanchor='center',
        xaxis=dict(title='Tipo de Perícia', categoryorder='total descending'),
        yaxis=dict(title='Quantidade'),
        hovermode='x unified',
        showlegend=False
    )

    st.plotly_chart(fig)
```

Figura 3.18: Método da classe *Gráfico* para o gráfico de coluna para porporção geral.

```python
def grafico_barras_horizontal_tipoPericia(self, colunas, cores):
    """
    Cria e exibe um gráfico de barras para a proporção geral comparando os tipos de atendimento.

    Parâmetros:
        colunas (list): lista com os nomes das colunas [categoria, valor].
        cores (list): cores de cada pericia.
    """
    categoria_col, valor_col = colunas

    # Definir cores especificas para cada categoria
    colors = {
        'SINGULAR': cores[0],
        'DOCUMENTAL': cores[1],
        'JUNTA': cores[2],
        'ODONTO': cores[3]
    }

    # Criação da figura
    fig = go.Figure()

    # Adicionar os dados ao gráfico
    fig.add_trace(go.Bar(
        x=self.df[valor_col],
        y=self.df[categoria_col],
        orientation='h',
        text=self.df[valor_col],
        textposition='auto',
        hovertemplate='<br>Tipo de Pericia: %{y}<br>Valor: %{x}',
        marker=dict(line=dict(color='Grey', width=2), color=[colors.get(cat, '#7f7f7f') for cat in self.df
        [categoria_col]])  # Usa as cores definidas
    ))

    # Configuração do layout
    fig.update_layout(
        title=f'Proporção Geral de Tipos de Pericia',
        title_x=0.5,
        title_xanchor='center',
        xaxis=dict(title='Quantidade'),
        yaxis=dict(title='Tipo de Pericia', categoryorder='total ascending'),
        hovermode='y unified',
        showlegend=False
    )

    st.plotly_chart(fig)
```

Figura 3.19: Método da classe *Gráfico* para o gráfico de barra para porporção geral.

Nessa versão do gráficos de pizza será comparado o mesmo tipo de atendimento mas em dois períodos diferentes, de acordo com o tipo de comparação selecionado.

```python
def grafico_pizza_periodos(self, colunas, tipoPericia, data_inicio, data_fim, periodo, cores):
    data_inicio_anterior = data_inicio - periodo
    data_fim_anterior = data_fim - periodo
    """
    Cria e exibe um gráfico de pizza comparando a soma de vallores de um mesmo tipo de pericia em dois periodos diferente.

    Parâmetros:
        colunas (list): lista com os nomes das colunas [ATUAL, ANTERIOR].
        cores (list): lista de cores para os respectivos tipos de pericia.
        data_inicio (datetime): data de inicio do periodo atual.
        data_fim (datetime): data de fim do periodo atual.
        periodo: diferença entre as datas inicial e final para que se tenha o periodo anterio com a mesma antidade de dias.
    """
    categoria_col, valor_col = colunas

    # Definir cores específicas para cada categoria
    colors = {
        'ATUAL': cores[0],
        'ANTERIOR': cores[1]
    }

    # Labels originais para o gráfico
    labels = list(self.df[categoria_col])

    # Construção de legendas personalizadas com periodos
    legenda_personalizada = []
    if [data_inicio, data_fim]:
        legenda_personalizada.append(f"ATUAL ({data_inicio} a {data_fim})")
    else:
        legenda_personalizada.append("ATUAL")

    if [data_inicio_anterior, data_fim_anterior]:
        legenda_personalizada.append(f"ANTERIOR ({data_inicio_anterior} a {data_fim_anterior})")
    else:
        legenda_personalizada.append("ANTERIOR")

    # Criação da figura do gráfico de pizza
    fig = go.Figure()

    fig.add_trace(go.Pie(
        labels=labels,  # Uso os rótulos originais no gráfico
        values=self.df[valor_col],
        hoverinfo='label+value',
        textinfo='label+percent',
        textfont=dict(size=14),
        hole=0.4,  # Gráfico tipo donut
        marker=dict(
            line=dict(color='Grey', width=2),
            colors=[colors.get(cat, '#7f7f7f') for cat in self.df[categoria_col]]
        ),
        showlegend=False  # Desativa a legenda padrão
    ))

    # Configuração do layout com legendas personalizadas
    fig.update_layout(
        title=f'{tipoPericia} - comparativo da quantidade de pericias do mesmo tipo entre periodos.',
        title_x=0.5,
        title_xanchor='center',
        showlegend=True,
        legend=dict(
            itemsizing='constant',
            title=dict(text="Periodos"),
            traceorder='normal'
        ),
        xaxis=dict(visible=False),  # Remove o eixo X
        yaxis=dict(visible=False),  # Remove o eixo Y
    )

    # Adicionando manualmente a legenda personalizada
    for i, label in enumerate(legenda_personalizada):
        fig.add_trace(go.Scatter(
            x=[None],  # Adiciona pontos invisíveis para a legenda
            y=[None],
            mode='markers',
            marker=dict(size=10, color=cores[i]),
            name=label
        ))

    st.plotly_chart(fig)
```

Figura 3.20: Método da classe *Gráfico* para o gráfico de pizza comparando períodos de um mesmo tipo de atendimento.

3.4.2 multGraficos

Com a necessidade dos métodos de gráficos serem passados para dois ou mais Dataframes, *multGraficos* foi projetado para receber uma lista. Com essa classe é possível exibir gráficos comparando dois períodos diferentes de um mesmo tipo de atendimento e, além de mostrar seus respectivos valores, podemos mostrar cada valor com sua data ao invés de mostrar apenas as datas referentes ao periodo atual selecionado. Os métodos da classe incluem a geração de gráficos de linha, colunas e barras horizontais.

O construtor da classe recebe como parâmetro uma lista de DataFrames. Essa lista pode conter, por exemplo, um DataFrame representando um período atual e outro representando um período anterior. O uso de self.dataframes permite armazenar esses dados na instância do objeto, tornando-os acessíveis a todos os métodos da classe.

Na figura 3.21 está o construtor da classe. A seguir, nas figuras 3.22, 3.23, 3.24 e 3.25 estão os métodos implementados.

```python
class MultGrafico:

    """
    Classe responsável por criar múltiplos gráficos comparativos usando um ou mais
    DataFrames.
    Permite gerar gráficos de linhas, colunas, barras horizontais e pizza.

    Métodos:
        - grafico_linha_PG: Cria gráficos de linha para a proporção geral mostrando uma
        linha para cada tipo de perícia.
        - grafico_linha: Cria gráficos de linha comparando um mesmo tipo de perícia em
        períodos diferentes.
        - grafico_coluna: Cria gráficos de colunas verticais comparando um mesmo tipo de
        perícia em períodos diferentes.
        - grafico_barras_horizontal: Cria gráficos de barras horizontais comparando um mesmo
        tipo de perícia em períodos diferentes.
        - grafico_pizza: Cria gráficos de pizza comparando dois valores referentes a somas
        de dois periodos de um mesmo tipo de perícia.

    Atributos:
        dataframes (list): Lista de pandas.DataFrame contendo os dados a serem usados nos
        gráficos.
    """

    def __init__(self, dataframes: list):
        """
        Inicializa a classe com uma lista de DataFrames.

        Args:
            dataframes (list): Lista de pandas.DataFrame.
        """
        self.dataframes = dataframes
```

Figura 3.21: Construtor da classe *MultGráfico*.

```python
    def grafico_linha_PG(self, colunas, tipoPericia, cores):
        """
        Cria um gráfico de linha para múltiplos DataFrames.

        Args:
            colunas (tuple): Colunas para o eixo x e y.
            tipoPericia (str): Título do gráfico.
            cores (list): Lista de cores para o conjunto mais recente.
        """
        x_col, y_col = colunas
        fig = go.Figure()

        nomes = ["Singular", "Documental", "Junta", "Odonto"]
        for i, df in enumerate(self.dataframes):
            fig.add_trace(go.Scatter(
                x=df[x_col],
                y=df[y_col],
                mode='lines+markers',
                name=f'{nomes[i]}',
                text=df[x_col],
                hovertemplate='<br>Data: %{text}<br>Valor: %{y}',
                line=dict(color=cores[i])
            ))

        fig.update_layout(
            title=f'{tipoPericia} - perícias ao longo do tempo.',
            title_x=0.1,
            title_xanchor='left',
            title_y=0.85,
            xaxis_title='Data',
            yaxis_title='Quant. de perícias',
            hovermode='x unified',
            yaxis=dict(tickformat='.'),
            legend=dict(x=1, y=1, xanchor='right', yanchor='bottom', font=dict(size=10))
        )

        st.plotly_chart(fig)
```

Figura 3.22: Método da classe *MultGráfico* para o gráfico de linha de proporção geral.

```python
def grafico_linha(self, colunas, data_inicio, data_fim, periodo_anterior, tipoPericia, cores):
    """
    Gera um gráfico de linha comparando dois períodos de uma mesma base de dados separados em dois dataframes.

    Args:
        colunas (tuple): Tuple com nomes das colunas para o eixo x e y.
        data_inicio (datetime): Data de início do período atual.
        data_fim (datetime): Data de fim do período atual.
        periodo_anterior (timedelta): Período anterior para comparação.
        tipoPericia (str): Tipo de perícia a ser exibida no título.
        cores (list): Lista contendo duas cores [cor1, cor2].
    """
    x_col, y_col = colunas
    fig = go.Figure()

    # Verifica se temos exatamente dois DataFrames
    if len(self.dataframes) != 2:
        raise ValueError("A lista de DataFrames deve conter exatamente dois DataFrames.")

    # Define o DataFrame do período atual
    df_atual = self.dataframes[0]
    df_comparacao = self.dataframes[1]

    # Adiciona a linha para o período atual
    fig.add_trace(go.Scatter(
        x=df_atual[x_col],
        y=df_atual[y_col],
        mode='lines+markers',
        name=f'Período atual: {data_inicio} a {data_fim}',
        text=df_atual[x_col],
        hovertemplate='<br>Data: %{text}<br>Valor: %{y}',
        line=dict(color=cores[0])
    ))

    # Adiciona a linha para o período de comparação
    fig.add_trace(go.Scatter(
        x=df_atual[x_col],  # Mantém as datas do período atual no eixo x
        y=df_comparacao[y_col],
        mode='lines+markers',
        name=f'Período anterior: {data_inicio-periodo_anterior} a {data_fim-periodo_anterior}',
        text=df_comparacao[x_col],
        hovertemplate='<br>Data: %{text}<br>Valor: %{y}',
        line=dict(color=cores[1])
    ))

    # Configura o layout
    fig.update_layout(
        title=f'{tipoPericia} - Comparação com início em {periodo_anterior.days} dias anteriores.',
        title_x=0.1,
        title_xanchor='left',
        title_y=0.85,
        xaxis_title='Data (Período Atual)',
        yaxis_title='Quant. de perícias',
        hovermode='x unified',
        yaxis=dict(tickformat='.'),
        legend=dict(x=1, y=1, xanchor='right', yanchor='bottom', font=dict(size=10))
    )

    st.plotly_chart(fig)
```

Figura 3.23: Método da classe *MultGráfico* para o gráfico de linha.

```python
def grafico_coluna(self, colunas, data_inicio, data_fim, periodo_anterior, tipoPericia, cores):
    """
    Gera um gráfico de colunas comparando dois períodos de uma mesma base de dados separados em dois dataframes.

    Args:
        colunas (tuple): Tuple com nomes das colunas para o eixo x e y (x_col, y_col).
        data_inicio (datetime): Data de início do período atual.
        data_fim (datetime): Data de fim do período atual.
        periodo_anterior (timedelta): Período anterior para comparação.
        tipoPericia (str): Tipo de perícia a ser exibida no título.
        cores (list): Lista contendo duas cores [cor1, cor2].
    """
    x_col, y_col = colunas
    fig = go.Figure()

    # Verifica se temos exatamente dois DataFrames
    if len(self.dataframes) != 2:
        raise ValueError("A lista de DataFrames deve conter exatamente dois DataFrames.")

    # Define o DataFrame do período atual
    df_atual = self.dataframes[0]
    df_comparacao = self.dataframes[1]

    # Adiciona as colunas para o período de comparação
    fig.add_trace(go.Bar(
        x=df_atual[x_col],  # Mantém as datas do período atual no eixo x
        y=df_comparacao[y_col],
        name=f'Período anterior: {data_inicio - periodo_anterior} a {data_fim - periodo_anterior}',
        text=df_comparacao[y_col],
        textposition='outside',
        customdata = df_comparacao[x_col],
        hovertemplate='<br>Data: %{customdata}<br>Valor: %{y}',
        marker=dict(line=dict(color='Grey', width=2), color=cores[1])
    ))

    # Adiciona as colunas para o período atual
    fig.add_trace(go.Bar(
        x=df_atual[x_col],
        y=df_atual[y_col],
        name=f'Período atual: {data_inicio} a {data_fim}',
        text=df_atual[y_col],
        textposition='outside',
        hovertemplate='<br>Data: %{x}<br>Valor: %{y}',
        marker=dict(line=dict(color='Grey', width=2), color=cores[0])
    ))

    # Configura o layout
    fig.update_layout(
        title=f'{tipoPericia} - Comparação com início em {periodo_anterior.days} dias anteriores.',
        title_x=0.1,
        title_xanchor='left',
        title_y=0.85,
        xaxis_title='Data',
        yaxis_title='Quant. de perícias',
        hovermode='x unified',
        yaxis=dict(tickformat='.'),
        xaxis=dict(title='Data', dtick=1, type='category'),
        legend=dict(x=1, y=1, xanchor='right', yanchor='bottom', font=dict(size=10)),
        barmode='group'
    )

    st.plotly_chart(fig)
```

Figura 3.24: Método da classe *MultGráfico* para o gráfico de coluna.

```python
def grafico_barras_horizontal(self, colunas, data_inicio, data_fim, periodo_anterior, tipoPericia, cores):
    """
    Gera um gráfico de barras horizontais comparando dois períodos de uma mesma base de dados separados em dois
    dataframes.

    Args:
        colunas (tuple): Tuple com nomes das colunas para o eixo x e y (x_col, y_col).
        data_inicio (datetime): Data de inicio do período atual.
        data_fim (datetime): Data de fim do período atual.
        periodo_anterior (timedelta): Período anterior para comparação.
        tipoPericia (str): Tipo de perícia a ser exibida no título.
        cores (list): Lista contendo duas cores [cor1, cor2].
    """
    x_col, y_col = colunas
    fig = go.Figure()

    # Verifica se temos exatamente dois DataFrames
    if len(self.dataframes) != 2:
        raise ValueError("A lista de DataFrames deve conter exatamente dois DataFrames.")

    # Define o DataFrame do período atual
    df_atual = self.dataframes[0]
    df_comparacao = self.dataframes[1]

    # Adiciona as barras horizontais para o período de comparação
    fig.add_trace(go.Bar(
        x=df_comparacao[y_col],
        y=df_atual[x_col],  # Mantém as categorias do período atual no eixo y
        orientation='h',
        name=f'Período anterior: {data_inicio - periodo_anterior} a {data_fim - periodo_anterior}',
        text=df_comparacao[y_col],
        textposition='outside',
        customdata = df_comparacao[x_col],
        hovertemplate='<br>Data: %{customdata}<br>Valor: %{x}',
        marker=dict(line=dict(color='Grey', width=2), color=cores[1])
    ))

    # Adiciona as barras horizontais para o período atual
    fig.add_trace(go.Bar(
        x=df_atual[y_col],
        y=df_atual[x_col],
        orientation='h',
        name=f'Período atual: {data_inicio} a {data_fim}',
        text=df_atual[y_col],
        textposition='outside',
        hovertemplate='<br>Data: %{y}<br>Valor: %{x}',
        marker=dict(line=dict(color='Grey', width=2), color=cores[0])
    ))

    # Configura o layout
    fig.update_layout(
        title=f'{tipoPericia} - Comparação com inicio em {periodo_anterior.days} dias anteriores.',
        title_x=0.1,
        title_xanchor='left',
        title_y=0.85,
        xaxis_title='Quant. de pericias',
        yaxis_title='Data',
        hovermode='y unified',
        xaxis=dict(tickformat=',.'),
        yaxis=dict(title='Data', dtick=1, type='category'),
        legend=dict(x=1, y=1, xanchor='right', yanchor='bottom', font=dict(size=10)),
        barmode='group'
    )

    st.plotly_chart(fig)
```

Figura 3.25: Método da classe *MultGráfico* para o gráfico de barra.

3.5 Tratamento de dados

O trecho de código mostrado na figura 3.26 tem a função de filtrar, processar e estruturar os dados garantindo que a visualização deles seja clara. O processo ocorre em algumas etapas principais, que incluem a filtragem dos dados, a contagem de ocorrências por data, o preenchimento de lacunas temporais, e a formatação dos dados para a geração de gráficos.

```python
#JUNTA
df_agendaJunta = df_agendaFiltrado[~df_agendaFiltrado['Título'].str.contains('SINGULAR|DOCUMENTAL|ODONTO|PROC|FALTOU|
faltou|Faltou|REMARCADO|remarcado|REMARCADA|remarcada|REMARCAR|remarcar|ACL|INICIO|inicio|às|ÀS|CADA|AGENDAR|TROCAR|
Trocar|Troca|TROCA|Folga|FOLGA|Ferias|FERIAS|Férias|FÉRIAS', case=False, na=False)]

df_agendaJunta_table = df_agendaJunta
df_agendaJunta_table = graficos.Grafico(df_agendaJunta_table)

df_agendaJunta = df_agendaJunta['Data Início'].value_counts().reset_index().sort_values(by='Data Início',
ascending=True)

#preenche as datas sem perícia com zeros para uma melhor visualização dos dados no período selecionado
df_agendaJunta = FD.preencher_datas_faltantes(df_agendaJunta, data_inicio, data_fim, "Data Início", "count")

df_juntab4ClassGrafico = df_agendaJunta

df_agendaJunta = graficos.Grafico(df_agendaJunta)
```

Figura 3.26: Exemplo de tratamento de dados.

Inicialmente, a variável df_agendaJunta é criada a partir do DataFrame filtrado, removendo registros que contenham palavras-chave específicas na coluna "Título". Essa filtragem é feita com a função .str.contains(), que verifica se a string da coluna contém qualquer uma dessas palavras e, caso contenha, a linha correspondente é removida (˜ operador de

negação). O parâmetro case=False torna a busca insensível a maiúsculas e minúsculas, e na=False evita erros ao lidar com valores nulos.

Após essa filtragem, os dados de df_agendaJunta são armazenados em df_agendaJunta_table, que é passado para um objeto da classe Grafico, permitindo que os dados sejam manipulados para visualização. Em seguida, o código contabiliza as ocorrências de atendimentos por data, utilizando .value_counts(), que conta o número de ocorrências de cada data presente na coluna "Data Início", e reorganiza esses valores em ordem cronológica com .sort_values(by='Data Início', ascending=True), garantindo que as datas sejam exibidas de forma crescente.

Um ponto importante na preparação dos dados é o preenchimento de datas ausentes, realizado pela função FD.preencher_datas_faltantes(). Como alguns dias podem não ter registros de perícias, essa função garante que todas as datas dentro do período selecionado estejam presentes no DataFrame, preenchendo os dias ausentes com 0 na contagem de atendimentos. Isso é essencial para manter a integridade visual dos gráficos, evitando que a ausência de registros crie "buracos"na visualização.

Após essa etapa de preenchimento, os dados processados são armazenados na variável df_juntab4ClassGrafico, preservando o conjunto estruturado antes de sua conversão para um objeto da classe Grafico. Finalmente, df_agendaJunta é transformado em um objeto gráfico (graficos.Grafico(df_agendaJunta)), o que indica que os dados estão prontos para receberem os métodos e os gráficos serem gerados.

Para os outros tipos de atendimento seguimos com essa mesma metodologia de tratamento de dados, mas com palavras-chave específicas para cada um.

3.6 Mapeamento de combinações

Para organaizar as cobinações foi utilizado um dicionário de mapeamento de funções (figuras 3.27, 3.28, 3.29, 3.30 e 3.31), que associa combinações específicas de opções selecionadas pelo usuário (figura 3.1) a métodos responsáveis por gerar gráficos ou tabelas. Esse dicionário funciona como uma estrutura centralizada que evita a necessidade de múltiplas condições if-else, tornando o código mais organizado e eficiente. Ele recebe como chaves tuplas de valores, que representam combinações de parâmetros como tipo de perícia, tipo de análise, formato do gráfico e tipo de comparação temporal. Cada chave é associada a uma função lambda, que executa a geração do gráfico correspondente por meio dos métodos das classes Grafico e MultGrafico.

O funcionamento se dá pela combinação de valores escolhidos pelo usuário na interface do sistema. Por exemplo, se um usuário seleciona "Junta"como tipo de perícia, "Atendimento"como análise, "Linha"como tipo de gráfico e "Nenhum"como método de comparação, o dicionário aciona a função df_agendaJunta.grafico_linha(), que gera o gráfico de linha correspondente para esse conjunto de filtros. Caso o usuário escolha uma comparação com um período anterior, como "x dias anteriores", a função chamada será diferente, acionando um método da classe MultGrafico para exibir a comparação dos dados em diferentes períodos.

```
mapeamento_funcoes = {
    # Junta
    ("Junta", "Atendimento", "Linha", "Nenhum"): lambda: df_agendaJunta.grafico_linha(['Data Inicio','count'],data_inicio,data_fim, box_tipoDePericia, corJunta),
    ("Junta", "Atendimento", "Coluna", "Nenhum"): lambda: df_agendaJunta.grafico_coluna(['Data Inicio','count'],data_inicio,data_fim, box_tipoDePericia, corJunta),
    ("Junta", "Atendimento", "Barra", "Nenhum"): lambda: df_agendaJunta.grafico_barras_horizontal(['Data Inicio','count'],data_inicio,data_fim, box_tipoDePericia, corJunta),
    ("Junta", "Atendimento", "Pizza", "Nenhum"): lambda: df_agendaJunta.mostrar_mensagem_nao_aplicavel(),
    ("Junta", "Atendimento", "Tabela para download", "Nenhum"): lambda: df_agendaJunta_table.tabela_para_download(),

    ("Junta", "Atendimento", "Linha", "x dias anteriores"): lambda: df_JuntaComp.grafico_linha(['Data Início','count'],data_inicio,data_fim, periodo_anterior,
    box_tipoDePericia, coresJunta_lista),
    ("Junta", "Atendimento", "Coluna", "x dias anteriores"): lambda: df_JuntaComp.grafico_coluna(['Data Inicio','count'],data_inicio,data_fim, periodo_anterior,
    box_tipoDePericia, coresJunta_lista),
    ("Junta", "Atendimento", "Barra", "x dias anteriores"): lambda: df_JuntaComp.grafico_barras_horizontal(['Data Inicio','count'],data_inicio,data_fim, periodo_anterior,
    box_tipoDePericia, coresJunta_lista),
    ("Junta", "Atendimento", "Pizza", "x dias anteriores"): lambda: df_JuntaPizza_anterior.grafico_pizza_periodos(['PERIODO','count'], box_tipoDePericia, data_inicio, data_fim,
    periodo_anterior, coresJunta_lista),
    ("Junta", "Atendimento", "Tabela para download", "x dias anteriores"): lambda: df_agendaJunta.mostrar_mensagem_nao_aplicavel(),

    ("Junta", "Atendimento", "Linha", "Período anterior"): lambda: df_JuntaComp_mesAnterior.grafico_linha(['Data Início','count'],data_inicio,data_fim, periodo_mesAnterior,
    box_tipoDePericia, coresJunta_lista),
    ("Junta", "Atendimento", "Coluna", "Período anterior"): lambda: df_JuntaComp_mesAnterior.grafico_coluna(['Data Início','count'],data_inicio,data_fim, periodo_mesAnterior,
    box_tipoDePericia, coresJunta_lista),
    ("Junta", "Atendimento", "Barra", "Período anterior"): lambda: df_JuntaComp_mesAnterior.grafico_barras_horizontal(['Data Início','count'],data_inicio,data_fim,
    periodo_mesAnterior, box_tipoDePericia, coresJunta_lista),
    ("Junta", "Atendimento", "Pizza", "Período anterior"): lambda: df_JuntaPizza_mesAnterior.grafico_pizza_periodos(['PERIODO','count'], box_tipoDePericia, data_inicio,
    data_fim, periodo_mesAnterior,coresJunta_lista),
    ("Junta", "Atendimento", "Tabela para download", "Período anterior"): lambda: df_agendaJunta.mostrar_mensagem_nao_aplicavel(),
```

Figura 3.27: Mapeamento da combinações para os gráficos de junta médica.

```
    # Singular
    ("Singular", "Atendimento", "Linha", "Nenhum"):  lambda: df_agendaSingular.grafico_linha(['Data Início','count'],data_inicio,data_fim, box_tipoDePericia, corSingular),
    ("Singular", "Atendimento", "Coluna", "Nenhum"): lambda: df_agendaSingular.grafico_coluna(['Data Inicio','count'],data_inicio,data_fim, box_tipoDePericia, corSingular),
    ("Singular", "Atendimento", "Barra", "Nenhum"): lambda: df_agendaSingular.grafico_barras_horizontal(['Data Inicio','count'],data_inicio,data_fim, box_tipoDePericia,
    corSingular),
    ("Singular", "Atendimento", "Pizza", "Nenhum"): lambda: df_agendaSingular.mostrar_mensagem_nao_aplicavel(),
    ("Singular", "Atendimento", "Tabela para download", "Nenhum"): lambda: df_agendaSingular_table.tabela_para_download(),

    ("Singular", "Atendimento", "Linha", "x dias anteriores"): lambda: df_SingularComp.grafico_linha(['Data Início','count'],data_inicio,data_fim, periodo_anterior,
    box_tipoDePericia, coresSingular_lista),
    ("Singular", "Atendimento", "Coluna", "x dias anteriores"): lambda: df_SingularComp.grafico_coluna(['Data Início','count'],data_inicio,data_fim, periodo_anterior,
    box_tipoDePericia, coresSingular_lista),
    ("Singular", "Atendimento", "Barra", "x dias anteriores"): lambda: df_SingularComp.grafico_barras_horizontal(['Data Inicio','count'],data_inicio,data_fim, periodo_anterior,
    box_tipoDePericia, coresSingular_lista),
    ("Singular", "Atendimento", "Pizza", "x dias anteriores"): lambda: df_SingularPizza_anterior.grafico_pizza_periodos(['PERIODO','count'], box_tipoDePericia, data_inicio,
    data_fim, periodo_anterior,coresSingular_lista),
    ("Singular", "Atendimento", "Tabela para download", "x dias anteriores"): lambda: df_agendaSingular.mostrar_mensagem_nao_aplicavel(),

    ("Singular", "Atendimento", "Linha", "Período anterior"): lambda: df_SingularComp_mesAnterior.grafico_linha(['Data Início','count'],data_inicio,data_fim,
    periodo_mesAnterior, box_tipoDePericia, coresSingular_lista),
    ("Singular", "Atendimento", "Coluna", "Período anterior"): lambda: df_SingularComp_mesAnterior.grafico_coluna(['Data Início','count'],data_inicio,data_fim,
    periodo_mesAnterior, box_tipoDePericia, coresSingular_lista),
    ("Singular", "Atendimento", "Barra", "Período anterior"): lambda: df_SingularComp_mesAnterior.grafico_barras_horizontal(['Data Inicio','count'],data_inicio,data_fim,
    periodo_mesAnterior, box_tipoDePericia, coresSingular_lista),
    ("Singular", "Atendimento", "Pizza", "Período anterior"): lambda: df_SingularPizza_mesAnterior.grafico_pizza_periodos(['PERIODO','count'], box_tipoDePericia, data_inicio,
    data_fim, periodo_mesAnterior,coresSingular_lista),
    ("Singular", "Atendimento", "Tabela para download", "Período anterior"): lambda: df_agendaSingular.mostrar_mensagem_nao_aplicavel(),
```

Figura 3.28: Mapeamento da combinações para os gráficos de perícia singular.

```
    # Proporção geral
    ("Proporção geral", "Atendimento", "Linha", "Nenhum"): lambda: df_propGeral_paralinha.grafico_linha_PG(['Data Início','count'], box_tipoDePericia, cores),
    ("Proporção geral", "Atendimento", "Coluna", "Nenhum"): lambda: df_propGeral.grafico_coluna_tipoPericia(['TIPO PERICIA','count'], cores),
    ("Proporção geral", "Atendimento", "Barra", "Nenhum"): lambda: df_propGeral.grafico_barras_horizontal_tipoPericia(['TIPO PERICIA','count'], cores),
    ("Proporção geral", "Atendimento", "Pizza", "Nenhum"): lambda: df_propGeral.grafico_pizza(['TIPO PERICIA','count'], cores),
    ("Proporção geral", "Atendimento", "Tabela para download", "Nenhum"): lambda: df_propGeral.tabela_para_download(),

    ("Proporção geral", "Atendimento", "Linha", "x dias anteriores"): lambda: df_propGeral.mostrar_mensagem_nao_aplicavel(),
    ("Proporção geral", "Atendimento", "Coluna", "x dias anteriores"): lambda: df_propGeral.mostrar_mensagem_nao_aplicavel(),
    ("Proporção geral", "Atendimento", "Barra", "x dias anteriores"): lambda: df_propGeral.mostrar_mensagem_nao_aplicavel(),
    ("Proporção geral", "Atendimento", "Pizza", "x dias anteriores"): lambda: df_propGeral.mostrar_mensagem_nao_aplicavel(),
    ("Proporção geral", "Atendimento", "Tabela para download", "x dias anteriores"): lambda: df_propGeral.mostrar_mensagem_nao_aplicavel(),

    ("Proporção geral", "Atendimento", "Linha", "Período anterior"): lambda: df_propGeral.mostrar_mensagem_nao_aplicavel(),
    ("Proporção geral", "Atendimento", "Coluna", "Período anterior"): lambda: df_propGeral.mostrar_mensagem_nao_aplicavel(),
    ("Proporção geral", "Atendimento", "Barra", "Período anterior"): lambda: df_propGeral.mostrar_mensagem_nao_aplicavel(),
    ("Proporção geral", "Atendimento", "Pizza", "Período anterior"): lambda: df_propGeral.mostrar_mensagem_nao_aplicavel(),
    ("Proporção geral", "Atendimento", "Tabela para download", "Período anterior"): lambda: df_propGeral.mostrar_mensagem_nao_aplicavel(),
```

Figura 3.29: Mapeamento da combinações para os gráficos de porporção geral.

```
# Documental
("Documental", "Atendimento", "Linha", "Nenhum"): lambda: df_agendaDocumental.grafico_linha(['Data Início','count'],data_inicio,data_fim, box_tipoDePericia, corDocumental),
("Documental", "Atendimento", "Coluna", "Nenhum"): lambda: df_agendaDocumental.grafico_coluna(['Data Início','count'],data_inicio,data_fim, box_tipoDePericia,
corDocumental),
("Documental", "Atendimento", "Barra", "Nenhum"): lambda: df_agendaDocumental.grafico_barras_horizontal(['Data Início','count'],data_inicio,data_fim, box_tipoDePericia,
corDocumental),
("Documental", "Atendimento", "Pizza", "Nenhum"): lambda: df_agendaDocumental.mostrar_mensagem_nao_aplicavel(),
("Documental", "Atendimento", "Tabela para download", "Nenhum"): lambda: df_agendaDocumental_table.tabela_para_download(),

("Documental", "Atendimento", "Linha", "x dias anteriores"): lambda: df_DocumentalComp.grafico_linha(['Data Início','count'],data_inicio,data_fim, periodo_anterior,
box_tipoDePericia, coresDocumental_lista),
("Documental", "Atendimento", "Coluna", "x dias anteriores"): lambda: df_DocumentalComp.grafico_coluna(['Data Início','count'],data_inicio,data_fim, periodo_anterior,
box_tipoDePericia, coresDocumental_lista),
("Documental", "Atendimento", "Barra", "x dias anteriores"): lambda: df_DocumentalComp.grafico_barras_horizontal(['Data Início','count'],data_inicio,data_fim,
periodo_anterior, box_tipoDePericia, coresDocumental_lista),
("Documental", "Atendimento", "Pizza", "x dias anteriores"): lambda: df_DocumentalPizza_anterior.grafico_pizza_periodos(['PERIODO','count'], box_tipoDePericia, data_inicio,
data_fim, periodo_anterior,coresDocumental_lista),
("Documental", "Atendimento", "Tabela para download", "x dias anteriores"): lambda: df_agendaDocumental.mostrar_mensagem_nao_aplicavel(),

("Documental", "Atendimento", "Linha", "Período anterior"): lambda: df_DocumentalComp_mesAnterior.grafico_linha(['Data Início','count'],data_inicio,data_fim,
periodo_mesAnterior, box_tipoDePericia, coresDocumental_lista),
("Documental", "Atendimento", "Coluna", "Período anterior"): lambda: df_DocumentalComp_mesAnterior.grafico_coluna(['Data Início','count'],data_inicio,data_fim,
periodo_mesAnterior, box_tipoDePericia, coresDocumental_lista),
("Documental", "Atendimento", "Barra", "Período anterior"): lambda: df_DocumentalComp_mesAnterior.grafico_barras_horizontal(['Data Início','count'],data_inicio,data_fim,
periodo_mesAnterior, box_tipoDePericia, coresDocumental_lista),
("Documental", "Atendimento", "Pizza", "Período anterior"): lambda: df_DocumentalPizza_mesAnterior.grafico_pizza_periodos(['PERIODO','count'], box_tipoDePericia,
data_inicio, data_fim, periodo_mesAnterior,coresDocumental_lista),
("Documental", "Atendimento", "Tabela para download", "Período anterior"): lambda: df_agendaDocumental.mostrar_mensagem_nao_aplicavel(),
```

Figura 3.30: Mapeamento da combinações para os gráficos de perícia documental.

```
# Odontológica
("Odontológica", "Atendimento", "Linha", "Nenhum"): lambda: df_agendaOdonto.grafico_linha(['Data Início','count'],data_inicio,data_fim, box_tipoDePericia, corOdonto),
("Odontológica", "Atendimento", "Coluna", "Nenhum"): lambda: df_agendaOdonto.grafico_coluna(['Data Início','count'],data_inicio,data_fim, box_tipoDePericia, corOdonto),
("Odontológica", "Atendimento", "Barra", "Nenhum"): lambda: df_agendaOdonto.grafico_barras_horizontal(['Data Início','count'],data_inicio,data_fim, box_tipoDePericia,
corOdonto),
("Odontológica", "Atendimento", "Pizza", "Nenhum"): lambda: df_agendaOdonto.mostrar_mensagem_nao_aplicavel(),
("Odontológica", "Atendimento", "Tabela para download", "Nenhum"): lambda: df_agendaOdonto_table.tabela_para_download(),

("Odontológica", "Atendimento", "Linha", "x dias anteriores"): lambda: df_OdontoComp.grafico_linha(['Data Início','count'],data_inicio,data_fim, periodo_anterior,
box_tipoDePericia, coresodonto_lista),
("Odontológica", "Atendimento", "Coluna", "x dias anteriores"): lambda: df_OdontoComp.grafico_coluna(['Data Início','count'],data_inicio,data_fim, periodo_anterior,
box_tipoDePericia, coresodonto_lista),
("Odontológica", "Atendimento", "Barra", "x dias anteriores"): lambda: df_OdontoComp.grafico_barras_horizontal(['Data Início','count'],data_inicio,data_fim,
periodo_anterior, box_tipoDePericia, coresodonto_lista),
("Odontológica", "Atendimento", "Pizza", "x dias anteriores"): lambda: df_OdontoPizza_anterior.grafico_pizza_periodos(['PERIODO','count'], box_tipoDePericia, data_inicio,
data_fim, periodo_anterior,coresodonto_lista),
("Odontológica", "Atendimento", "Tabela para download", "x dias anteriores"): lambda: df_agendaOdonto.mostrar_mensagem_nao_aplicavel(),

("Odontológica", "Atendimento", "Linha", "Período anterior"): lambda: df_OdontoComp_mesAnterior.grafico_linha(['Data Início','count'],data_inicio,data_fim,
periodo_mesAnterior, box_tipoDePericia, coresodonto_lista),
("Odontológica", "Atendimento", "Coluna", "Período anterior"): lambda: df_OdontoComp_mesAnterior.grafico_coluna(['Data Início','count'],data_inicio,data_fim,
periodo_mesAnterior, box_tipoDePericia, coresodonto_lista),
("Odontológica", "Atendimento", "Barra", "Período anterior"): lambda: df_OdontoComp_mesAnterior.grafico_barras_horizontal(['Data Início','count'],data_inicio,data_fim,
periodo_mesAnterior, box_tipoDePericia, coresodonto_lista),
("Odontológica", "Atendimento", "Pizza", "Período anterior"): lambda: df_OdontoPizza_mesAnterior.grafico_pizza_periodos(['PERIODO','count'], box_tipoDePericia, data_inicio,
data_fim, periodo_mesAnterior,coresodonto_lista),
("Odontológica", "Atendimento", "Tabela para download", "Período anterior"): lambda: df_agendaOdonto.mostrar_mensagem_nao_aplicavel(),
}
```

Figura 3.31: Mapeamento da combinações para os gráficos de perícia odontológica.

Esse mecanismo proporciona flexibilidade e escalabilidade ao sistema, pois permite adicionar novos tipos de gráficos ou análises sem modificar diretamente a lógica principal do programa. Como as funções são armazenadas dentro do dicionário, basta incluir novas combinações dentro da estrutura para expandir as funcionalidades sem comprometer a legibilidade e a eficiência do código. Além disso, a utilização de funções lambda garante que a execução de cada opção seja realizada apenas quando a combinação específica for selecionada pelo usuário, otimizando o desempenho da aplicação.

O trecho de código abaixo, na figura 3.32, tem a função de determinar qual gráfico ou tabela será gerado com base nas opções selecionadas pelo usuário na interface da aplicação. Ele faz isso verificando se a combinação das opções escolhidas está presente no dicionário de mapeamento de funções (mapeamento_funcoes). Esse processo garante que, ao selecionar diferentes filtros e formatos, o sistema direcione corretamente a execução para a função responsável por gerar a visualização correspondente.

```python
# Define a chave com base nas seleções do usuário
# As variáveis box_tipoDePericia, box_analise, box_tipoDeGrafico, e box_comparacao são presumivelmente valores
# selecionados por meio de widgets interativos no Streamlit
chave = (box_tipoDePericia, box_analise, box_tipoDeGrafico, box_comparacao)


# Verifica se a combinação da chave existe no dicionário de mapeamento
if chave in mapeamento_funcoes:
    # Executa a função correspondente à combinação de opções selecionada
    mapeamento_funcoes[chave]()
    pass
else:
    st.error("Opção de gráfico não disponível para a combinação selecionada.")
```

Figura 3.32: Trecho do algoritmo para determinar qual gráfico será exibido de acordo com a combinação.

O código começa definindo a variável chave, que armazena uma tupla contendo os valores selecionados pelo usuário nos widgets interativos do Streamlit, vistos na figura 3.1.

Em seguida, há uma verificação condicional para determinar se a chave gerada a partir das seleções do usuário existe dentro do dicionário de mapeamento. Caso a chave esteja presente (if chave in mapeamento_funcoes:), a função correspondente àquela combinação específica é executada utilizando mapeamento_funcoes[chave]():. Isso significa que o código chama dinamicamente a função previamente definida dentro do dicionário, sem precisar de estruturas if-else extensas para cada combinação possível.

Se a chave não existir no dicionário, o código executa o bloco else, exibindo uma mensagem de erro na interface do Streamlit com st.error("Opção de gráfico não disponível para a combinação selecionada."). Esse tratamento evita que a aplicação tente gerar gráficos inválidos, garantindo uma melhor experiência para o usuário e prevenindo erros inesperados.

Capítulo 4

Experimentos e Resultados

A implementação do sistema proposto para a Divisão de Atenção à Saúde do Servidor (DAS) da Universidade Federal do Rio Grande do Norte (UFRN) foi concluída com sucesso, atendendo aos requisitos estabelecidos no início do projeto. O sistema permitiu a automação do processo de visualização e análise de atendimentos médicos, consolidando informações em tempo real por meio da integração com o Google Sheets. A interface desenvolvida com Streamlit mostrou-se intuitiva e responsiva, possibilitando aos usuários a seleção de períodos de análise e a geração de gráficos detalhados sem necessidade de conhecimentos avançados em manipulação de dados.

Os resultados obtidos demonstram a eficiência do sistema na organização e apresentação de informações sobre os atendimentos. O uso de gráficos interativos, gerados com as bibliotecas Plotly e Matplotlib, permitiu uma melhor compreensão da distribuição dos atendimentos ao longo do tempo.

Além disso, a utilização do st.session_state garantiu a persistência das configurações do usuário, tornando a experiência de navegação mais fluida. A filtragem de dados, implementada por meio das funções filtroDeDatas e preencher_datas_faltantes, contribuiu para uma análise mais precisa, garantindo que nenhum dado fosse perdido durante a consulta. A opção de exportação de dados para CSV viabilizou o armazenamento e compartilhamento de informações, facilitando a comunicação interna da unidade.

O sistema demonstrou desempenho satisfatório, com tempos de resposta rápidos e interface intuitiva, cumprindo com êxito o objetivo de aprimorar a análise dos atendimentos médicos realizados na unidade SIASS.

A seguir estarão algumas imagens do sistema funcionando e alguns exemplos de combinações para gerar os gráficos.

Começando com a tela inicial (figura 4.1) e a barra lateral com o sistema de navegação (figura 4.2), desenvolvidos como mostrado nas 3.2 e 3.7, respectivamente.

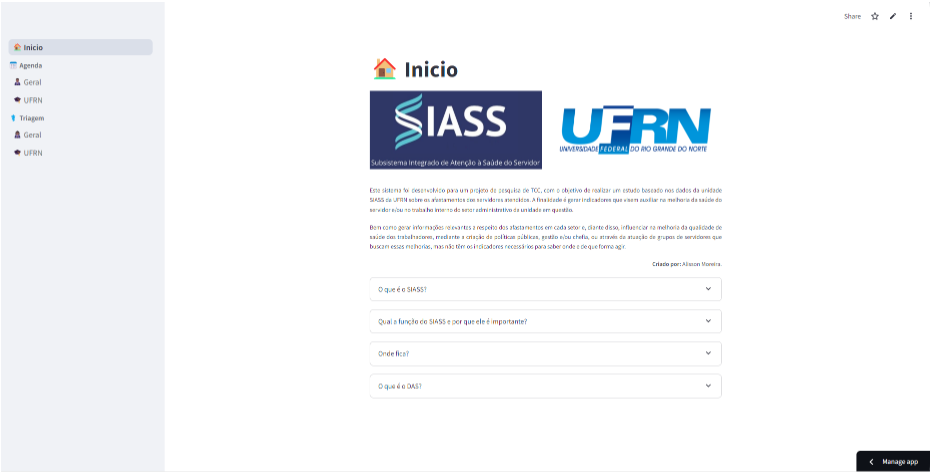


Figura 4.1: Tela inicial do sistema.

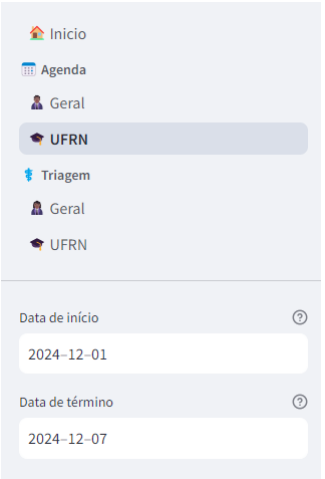


Figura 4.2: Barra lateral com sitema de navegação para multiplas páginas e um input de datas.

Na sequência estarão algumas figuras mostrando alguns gráficos e suas combinações como exemplo de funcionamento do sistema. As métodos que os implementam foram msotrados no capítulo 3, como pode ser visto nas figuras 3.11 a 3.25. Todos esses gráficos implementados posuem o método *hover*, que possíbilita passar o cursor por cima da informação para ter mais detalhes, como data do periodo atual e anterior, valores ou porcentagens que aquele dado representa.

O período utilizado foi de 01/12/ 2024 a 07/12/2024 e foi escolhido por ser antes dos recessos e ser uma semana onde ocorreram todos os tipos de atendimento.

Na figura 4.3 está um gráfico de linha para a proporção geral dos tipos de atendimento. Implementado com o algoritmo mostrado na figura 3.22.

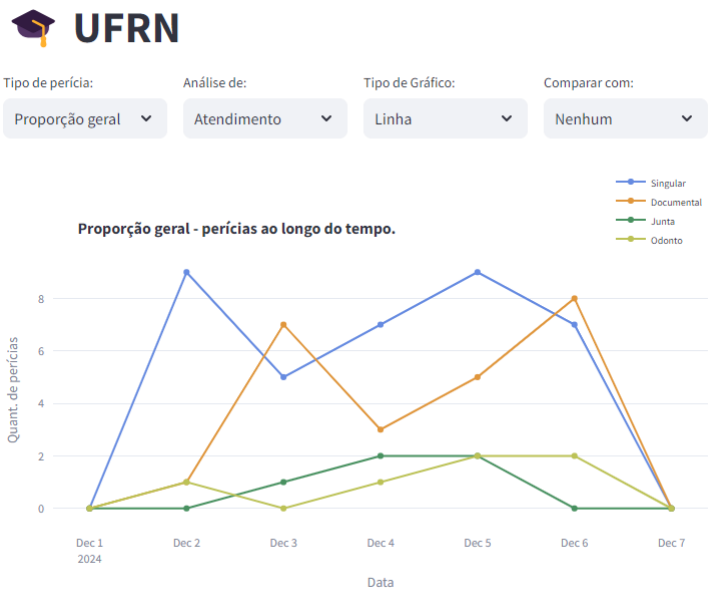


Figura 4.3: Gráfico de linha para a proporção geral dos atendimentos sem comparação com o periodo anterior.

Na figura 4.4 está um gráfico de linha para os atendimentos de perícia singular. Implementado com o algoritmo mostrado na figura 3.12.

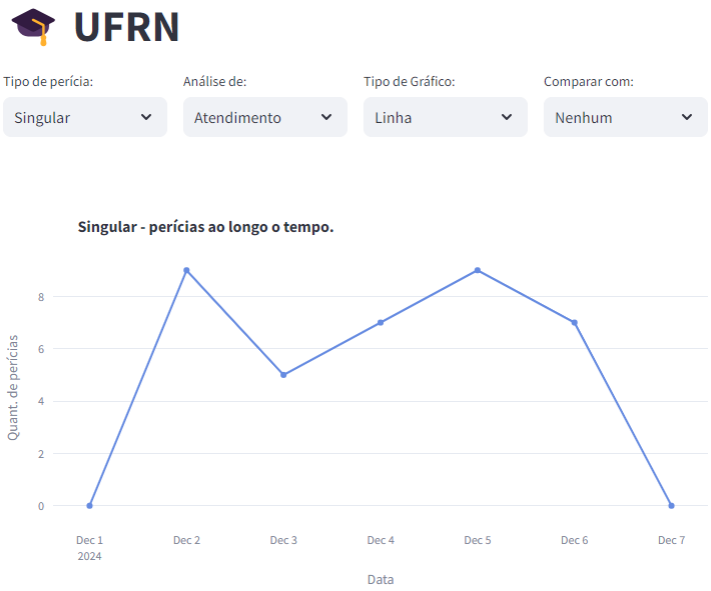


Figura 4.4: Gráfico de linha para atendimento de perícia singular sem comparação com o periodo anterior.

Na figura 4.5 está um gráfico de coluna para os atendimentos de perícia documental com uma comparação com "x"dias anteriores, onde esse "x"representa a quantidade de dias do período selecionado, como foi cidado no capítulo 3 quando foi falado sobre as funcionalidades. Implementado com o algoritmo mostrado na figura 3.24.

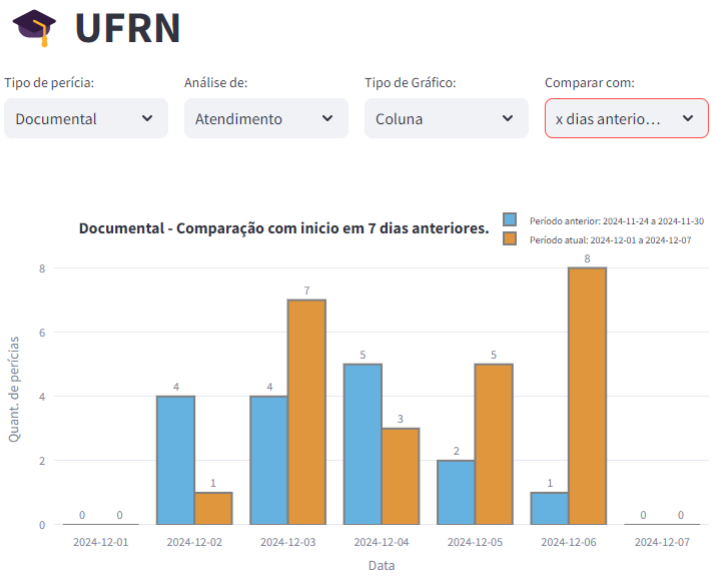


Figura 4.5: Gráfico de coluna para atendimento de perícia documental comparando com "x"dias anteriores.

Na figura 4.6 está um gráfico de coluna para os atendimentos de junta médica com uma comparação em relação ao mês anterior. A data com o mesmo dia no mês anterior é obtida através da função mostrada na figura 3.10. Implementado com o algoritmo mostrado na figura 3.24.

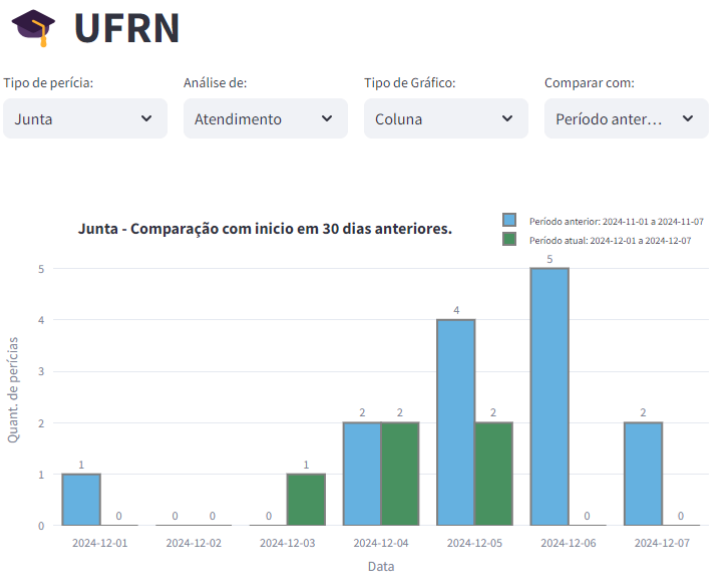


Figura 4.6: Gráfico de coluna para atendimento de junta médica comparando com o mês anterior.

Na figura 4.7 está um gráfico de barra para os atendimentos de perícia odontológica. Implementado com o algoritmo mostrado na figura 3.14.

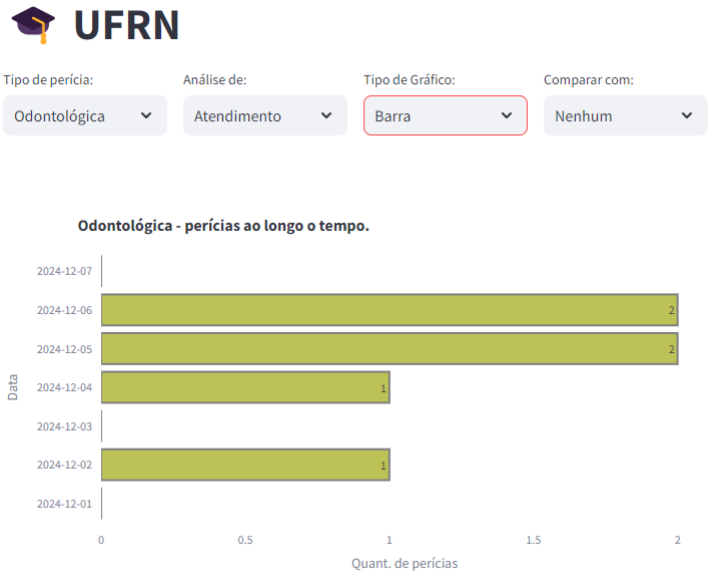


Figura 4.7: Gráfico de barra para atendimento de perícia odontológica sem comparação com periodo anterior.

Na figura 4.8 está um gráfico de pizza para os atendimentos de perícia singular comparando o periodo atual com "x"dias anteriores. Implementado com o algoritmo mostrado

na figura 3.20.

Figura 4.8: Gráfico de pizza para atendimento de perícia singular comparando com "x"dias anteriores.

Na figura 4.9 está um gráfico de linha para os atendimentos de perícia documental comparando o periodo atual com o mês anterior. Implementado com o algoritmo mostrado na figura 3.23.

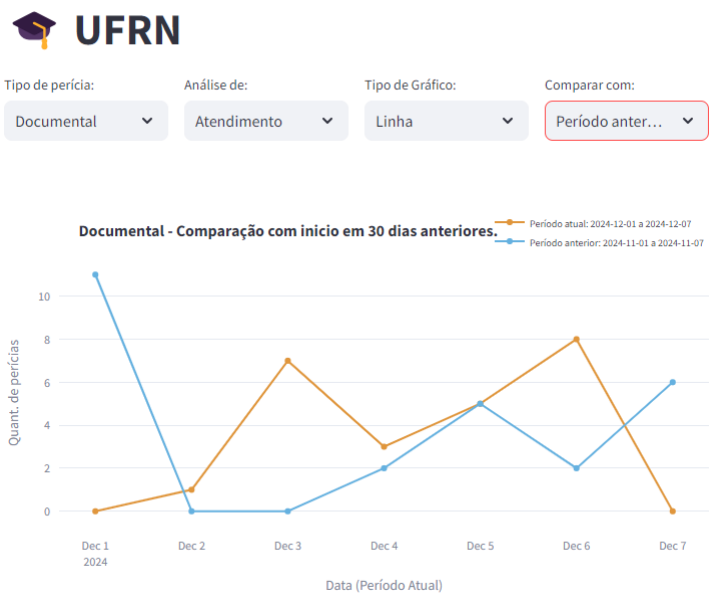
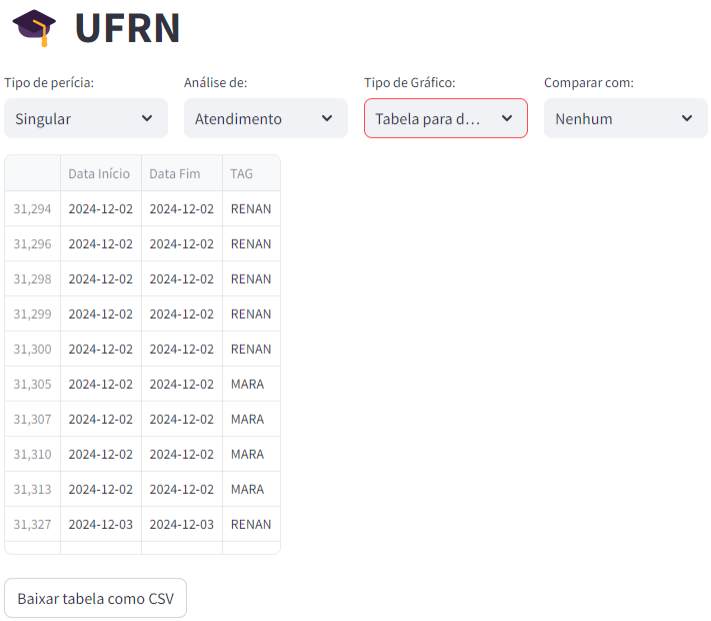


Figura 4.9: Gráfico de linha para atendimento de perícia documental comparando com o mês anterior.

Na figura 4.10 está uma tabela para os atendimentos de perícia singular mostrando data de atendimento e médicos que o realizaram. Implementado com o algoritmo mostrado na figura 3.15.



	Data Início	Data Fim	TAG
31,294	2024-12-02	2024-12-02	RENAN
31,296	2024-12-02	2024-12-02	RENAN
31,298	2024-12-02	2024-12-02	RENAN
31,299	2024-12-02	2024-12-02	RENAN
31,300	2024-12-02	2024-12-02	RENAN
31,305	2024-12-02	2024-12-02	MARA
31,307	2024-12-02	2024-12-02	MARA
31,310	2024-12-02	2024-12-02	MARA
31,313	2024-12-02	2024-12-02	MARA
31,327	2024-12-03	2024-12-03	RENAN

Figura 4.10: Tabela para atendimentos de perícia singular.

Na figura 4.11 mostra a mensagem exibida para combinações que não geram gráficos ou tabelas. Implementado com o algoritmo mostrado na figura 3.16.

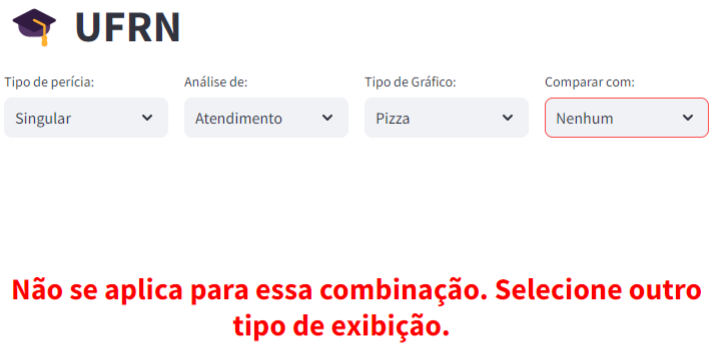


Figura 4.11: Mensagem exibida para combinações que não geram gráficos.

Capítulo 5

Conclusão

O desenvolvimento deste sistema representou um avanço significativo na gestão e visualização de dados dos atendimentos realizados no SIASS da UFRN. A implementação bem-sucedida de uma solução que combina Python, Streamlit e Google Sheets demonstrou que é possível integrar ferramentas modernas para automatizar a análise de informações e otimizar fluxos de trabalho administrativos.

Com a conclusão do projeto, verificou-se que o sistema atendeu aos requisitos estabelecidos, proporcionando um ambiente interativo para consulta e análise de atendimentos médicos. A interface desenvolvida facilitou a navegação e permitiu aos usuários selecionar períodos específicos para a geração de gráficos detalhados. A funcionalidade de comparação entre períodos tornou-se um diferencial na avaliação da demanda por perícias, permitindo que gestores identifiquem padrões de atendimento e otimizem a alocação de recursos.

A integração com o Google Sheets, utilizando a biblioteca Pygsheets, garantiu a atualização dinâmica dos dados, assegurando que as análises fossem sempre baseadas em informações atualizadas. A utilização de gráficos interativos melhorou a experiência dos usuários, proporcionando uma visualização clara e intuitiva dos dados processados.

Diante dos resultados alcançados, pode-se concluir que o sistema desenvolvido trouxe ganhos significativos na eficiência e transparência da análise dos atendimentos médicos, tornando-se uma ferramenta útil para a unidade SIASS da UFRN. A modularidade do sistema também permite futuras expansões, possibilitando a incorporação de novas funcionalidades conforme a necessidade do setor.

Como trabalhos futuros, sugere-se a implementação de novos métodos de análise estatística, além da possibilidade de integração com outros sistemas administrativos da universidade. A inclusão de relatórios automáticos e alertas para padrões atípicos nos atendimentos pode ampliar ainda mais o impacto positivo da ferramenta na gestão da saúde dos servidores da UFRN.

Referências Bibliográficas

[1] GOV.BR. Agência Nacional de Aviação Civil (Anac). *Manual de Perícia Oficial em Saúde do Servidor Público Federal*. 3. ed. Brasília: MP, 2017. Disponível em: https://www.gov.br/anac/pt-br/centrais-de-conteudo/publicacoes/arquivos/manual-de-pericia-oficial-em-saude-do-servidor-publico-federal-3a-edicao-ano-2017-versao-28abr2017-3.pdf/view. Acesso em: 12 jan. 2025.

[2] GOV.BR. Portal do Servidor. *Subsistema Integrado de Atenção à Saúde do Servidor (SIASS)*. Disponível em: https://www.gov.br/servidor/pt-br/siass. Acesso em: 12 jan. 2025.

[3] GOV.BR. Portal do Servidor. *O que é o SIASS*. Disponível em: https://www.gov.br/servidor/pt-br/siass/acesso_informacao/institucional/apresentacao-o-que-e-o-siass. Acesso em: 12 jan. 2025.

[4] PYTHON SOFTWARE FOUNDATION. *Python: Programming Language*. Disponível em: https://www.python.org/. Acesso em: 12 jan. 2025.

[5] STREAMLIT INC. *Streamlit: The fastest way to build and share data apps*. Disponível em: https://streamlit.io/. Acesso em: 12 jan. 2025.

[6] STREAMLIT INC. Streamlit Documentation. Disponível em: https://docs.streamlit.io. Acesso em: 12 jan. 2025.

[7] THE PANDAS DEVELOPMENT TEAM. Pandas Documentation. Disponível em: https://pandas.pydata.org/docs. Acesso em: 12 jan. 2025.

[8] BLACKARY. st_pages Documentation. Disponível em: https://github.com/blackary/st_pages. Acesso em: 12 jan. 2025

[9] PYGSHEETS. Pygsheets Documentation. Disponível em: https://pygsheets.readthedocs.io. Acesso em: 12 jan. 2025.

[10] THE PANDAS DEVELOPMENT TEAM. Pandas to_datetime Documentation. Disponível em: https://pandas.pydata.org/docs/reference/api/pandas.to_datetime.html. Acesso em: 12 jan. 2025.

[11] THE MATPLOTLIB DEVELOPMENT TEAM. Matplotlib Documentation. Disponível em: https://matplotlib.org/stable/index.html. Acesso em: 12 jan. 2025.

[12] PLOTLY TECHNOLOGIES INC. Plotly Documentation. Disponível em: https://plotly.com/python/. Acesso em: 12 jan. 2025.

[13] GOOGLE. Apps Script Documentation. Disponível em: https://developers.google.com/apps-script. Acesso em: 12 jan. 2025.

[14] PYTHON. Python Tutorial. Python Software Foundation, 2001. Disponível em: https://docs.python.org/3/tutorial/classes.html. Acesso em: 12 jan. 2025.

[15] PYLADIES BH. Gspread: Trabalhando com o Google Sheets. Medium, 15 mar. 2020. Disponível em: https://medium.com/pyladiesbh/gspread-trabalhando-com-o-google-sheets-f12e53ed1346. Acesso em: 12 jan. 2025.